DEVELOPMENT OF AN EARLY WARNING AND PREDICTION SYSTEM FOR LOCUST OUTBREAKS USING MACHINE LEARNING TECHNIQUES

ABDULLAHI ABDI HASHI

SULEIMAN ALI ABSHIR

AHMED IBRAHIM AHMED

AYAANLE AHMED ADOW

SUBMISSION OF GRADUATION PROJECT FOR PARTIAL FULFILLMENT OF THE DEGREE OF BACHELOR OF COMPUTER APPLICATIONS

JAMHURIYA UNIVERSITY OF SCIENCE AND TECHNOLOGY (JUST)

FACULTY OF COMPUTER & INFORMATION TECHNOLOGY

AUGUST 2025

ORIGINAL LITERARY WORK DECLARATION

Name of Candidate 1: ABDULLAHI ABDI HASHI **ID: C1210524**

Name of Candidate 2: AHMED IBRAHIM AHMED **ID: C1210484**

Name of Candidate 3: SULEIMAN ALI ABSHIR **ID: C1210450**

Name of Candidate 4: AYAANLE AHMED ADOW **ID: C1210568**

Name of Degree: Bachelor of Computer Application

Title of Project Paper/Research Report/Dissertation/Thesis ("this Work"):

DEVELOPMENT OF AN EARLY WARNING AND PREDICTION SYSTEM FOR LOCUST OUTBREAKS USING MACHINE LEARNING TECHNIQUES

Field of Study: Computer Applications

We the undersigned do solemnly and sincerely declare that:

(1) We are the authors/writers of this Work.

(2) This Work is original.

(3) Any use of any work in which copyright exists was done by way of fair dealing and for permitted purposes and any excerpt or extract from, or reference to or reproduction of any copyright work has been disclosed expressly and sufficiently and the title of the Work and its authorship have been acknowledged in this Work;

(4) We do not have any actual knowledge nor ought I reasonably to know that the making of this work constitutes an infringement of any copyright work.

(5) We hereby assign all and every right in the copyright to this Work to Jamhuriya University of Science and technology ("JUST"), who henceforth shall be owner of the copyright in this Work and that any reproduction or use in any form or by any means whatsoever is prohibited without the written consent of JUST having been first had and obtained.

(6) We are fully aware that if in the course of making this Work we have infringed any copyright whether intentionally or otherwise, we may be subject to legal action or any other action as may be determined by JUST.

Candidate 1"s Signature: _____, Candidate 2"Signature: _____

Candidate 3"s Signature: _____, Candidate 4"s Signature: _____

Date: _____ Subscribed and solemnly declared before,

Supervisor's Signature: _____

Date: _____

DEDICATION

We dedicate our dissertation work to our family and many friends. A special Feeling of gratitude to our loving parents, whose words of encouragement and push for tenacity ring in our ears.

ABSTRACT

This study focused on the development of a machine learning-based system for locust swarm prediction in Somalia, addressing the urgent challenges faced by farmers and policymakers due to unpredictable and destructive locust outbreaks. Traditional methods—such as manual field surveys and pesticide spraying—are often reactive, labor-intensive, and inadequate in scale and precision. The main objective of this research is to build a lightweight, interpretable, and user-friendly locust prediction system that leverages environmental data for early warning and decision support. The proposed system integrates Python, supervised machine learning algorithms (e.g., Logistic Regression, LightGBM, SVM, and Random Forest), and publicly available datasets from FAO. The methodology combines data preprocessing, model training, and frontend/backend development using tools like Flask, Jupyter Notebook, and MySQL. The system features a web-based interface that allows users to input environmental variables and receive near real-time predictions of locust presence, supported by visualizations and actionable insights. Key results demonstrate that the system achieves over 95.77% prediction accuracy with fast response times, significantly improving the efficiency and timeliness of locust monitoring compared to traditional practices. The research highlights the transformative potential of AI and geospatial data in agricultural resilience, especially in low-resource environments.

Keywords: Locust prediction, Machine Learning Models, Early warning system, Somalia, Environmental data, Agricultural resilience, Food security, Web-based application.

ACKNOWLEDGEMNTS

First and foremost, we are profoundly grateful to our powerful Allah [SWT], whose divine guidance and assistance made it possible for us to undertake and complete this task.

Secondly, we extend our heartfelt gratitude to our parents. Their unwavering support, both financially and morally, has been a cornerstone of our academic journey, providing us with the strength and determination needed to pursue and complete our studies.

Thirdly, we express our sincere appreciation to all our university teachers who have imparted their knowledge and wisdom to us. A special thank you goes to our supervisor, Mr. Abdirahman Omar Mohamud, for his exceptional guidance, patience, and support throughout this study. His invaluable insights and constructive feedback have been instrumental in shaping this work. **Fourthly**, we acknowledge and thank all our respondents who took the time to provide us with valuable answers. Their contributions were crucial in enriching the quality and depth of our research.

Finally, we extend our thanks to everyone who offered us advice or information during this journey. Each contribution has been greatly appreciated and has played a pivotal role in the successful completion of this work.

ALL BLESSINGS TO ALLAH.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

AI = Artificial Intelligence

CNN = Convolutional Neural Network

DFD = Data Flow Diagram

CSV = Comma-Separated Values

DLDM = Desert Locust Development Model

DLIS = Desert Locust Information Service

DLTM =Desert Locust Trajectory Model

DNN =Deep Neural Network

FAO =Food and Agriculture Organization

GIS =Geographic Information System

HMM =Hidden Markov Model

HSI =Habitat Suitability Index

HTML = HyperText Markup Language

IDE =Integrated Development Environment

IPM =Integrated Pest Management

IQR =Interquartile Range

JWT =JSON Web Token

k-NN =KNN k-Nearest Neighbors

LASSO =Least Absolute Shrinkage and Selection Operator

LGBM =LightGBM Light Gradient Boosting Machine

LSTM =Long Short-Term Memory

MaxEnt =Maximum Entropy Model

ML =Machine Learning

MySQL =My Structured Query Language (Database)

NDVI =Normalized Difference Vegetation Index

PCA =Principal Component Analysis

PLAN =Predictor of Locust Activity and movement

RNN =Recurrent Neural Network

SMOTE =Synthetic Minority Over-sampling Technique

SVM =Support Vector Machine

UAV =Unmanned Aerial Vehicle

VS Code =Visual Studio Code

WCEC =Website Content Extractor Chatbot

YOLO = You Only Look Once (Object Detection Algorithm)

CHAPTER I: INTRODUCTION

1.1 Background of The Study

Locust Swarm: A locust swarm is a large group of locusts that move together, often covering hundreds of square kilometres. These swarms can travel long distances and eat huge amounts of crops and vegetation in a short time, causing severe damage to agriculture and food supplies(Khan et al., 2024).

Mitigation: Mitigation refers to actions taken to reduce the impact of a problem. In the case of locust swarms, mitigation includes activities like spraying pesticides, using early warning systems, and implementing farming practices that make it harder for locusts to breed and spread (Showler et al., 2022).

Locust outbreaks are among the most destructive natural phenomena affecting agriculture worldwide. Desert locusts (Kimathi et al., 2020) are notorious for forming massive swarms capable of devastating crops and vegetation rapidly.

According to the Food and Agriculture Organization(FAO, 2021), a single square kilometre of a locust swarm can consume food equivalent to the daily intake of 35,000 people, posing severe threats to food security in vulnerable regions like Somalia. These outbreaks are exacerbated by climate variability and environmental degradation, which create favourable breeding conditions (Dinku et al., 2007).

The recurrence of locust outbreaks is driven by interconnected environmental factors. Climatic conditions such as irregular rainfall, temperature fluctuations, and humidity levels directly influence locust breeding cycles (Showler et al., 2022).

Human activities, including deforestation and unsustainable farming practices, further disrupt ecosystems, making locust prediction challenging (Zhang et al., 2022).

Traditional management strategies, such as pesticide spraying and manual surveys, are resource-intensive and often ineffective for large-scale infestations(FAO, 2021).

A key challenge is the lack of reliable early warning systems. Manual ground surveys and basic ecological models struggle to account for the dynamic behaviour of locust swarms(Tucker, 1985).

Economically, locust outbreaks devastate livelihoods. The 2020 invasion in East Africa caused an estimated $8.5 billion in agricultural damage, crippling smallholder farmers (FAO, 2020). Governments and NGOs often allocate limited resources to reactive measures like aerial pesticide spraying, which are costly and environmentally harmful (Showler et al., 2022). These challenges highlight the urgent need for cost-effective, proactive solutions.

Machine learning (ML) offers transformative potential for locust management. Unlike traditional models, ML algorithms can analyse large datasets—such as satellite imagery, weather patterns, and vegetation indices—to detect early signs of outbreaks (Marković et al., 2021). For example, deep learning models like convolutional neural networks (CNNs) have been used to identify locust breeding sites from remote sensing data (LeCun et al., 2015).

Similarly, recurrent neural networks (RNNs) can predict swarm migration paths by analysing historical climate trends (Zhang et al., 2022). These tools enable timely interventions, such as targeted pesticide application, reducing economic and environmental costs (FAO, 2020).

Despite ML's promise, gaps persist. Many regions lack high-quality datasets for training models, limiting their accuracy (Reichstein et al., 2019). Additionally, ML solutions must be interpretable for non-technical stakeholders, such as farmers and policymakers (Zhang et al., 2022).

Ethical concerns, including the environmental impact of ML-driven pesticide use, also require attention (Showler et al., 2022). Collaborative efforts between researchers, governments, and communities are critical to developing accessible, sustainable solutions for Somalia and other affected regions.

In rural Somalia, traditional locust control methods such as using smoke to deter swarms, making loud noises with pots and pans, and physically chasing or killing the insects were commonly practiced by local farmers and herders, especially in the absence of formal government support or modern tools(ICRC, 2022). These community-driven approaches reflect long-standing indigenous knowledge systems, often passed down through generations, and were born out of necessity in remote regions with limited infrastructure. However, while these methods provided some immediate relief, they were generally ineffective at controlling large-scale infestations. In contrast, recent interventions supported by the Food and Agriculture Organization (FAO) have introduced modern strategies such as the use of biopesticides like Metarhizium acridum, which target locusts specifically without harming other organisms or the environment. These biopesticide-based campaigns, implemented through coordinated efforts with Somali authorities and local partners, offer a more targeted, sustainable, and scientifically informed approach to locust management(FAO, 2022). The transition from traditional to modern methods highlights the evolving nature of pest control in Somalia, as communities increasingly gain access to more effective and environmentally responsible technologies.

1.2 Problem Statement

Locust outbreaks are among the most destructive natural events in agriculture worldwide, capable of devastating crops and vegetation within a short period and threatening the livelihoods of millions. A single swarm can travel vast distances, consuming food supplies equivalent to the daily needs of tens of thousands of people, thereby undermining food security and economic stability on a large scale. Their occurrence is influenced by environmental factors such as rainfall, temperature, and vegetation growth, making accurate prediction both complex and critical for effective prevention. Current control methods such as manual surveys, pesticide spraying, and basic ecological models are slow, resource-intensive, and often fail to provide timely alerts. In many affected regions, there is no robust early warning system that integrates real-time environmental data with predictive capabilities, leaving a significant gap in proactive locust management. This gap is especially evident in Somalia, where limited infrastructure and poor access to timely data hinder effective intervention, leading to recurrent crop losses, economic instability, and worsening food insecurity. Addressing this challenge is vital, as an effective prediction system could enable timely interventions, reduce the scale of damage, and promote sustainable agricultural practices. Therefore, this study aims to develop a machine learning based model for predicting locust outbreaks, providing early warnings to protect crops and strengthen food security in Somalia.

1.3 Research Objective

1.3.1 General Objective

The general objective of this study is to develop an advanced, machine learning-based framework for the early detection, prediction, and management of desert locust outbreaks, thereby reducing their adverse impacts on agriculture, food security, and economic stability.

1.3.2 Specific Objectives

1. To implement a machine learning model capable of accurately predicting locust outbreaks using publicly available environmental and agricultural datasets.
2. To evaluate the accuracy, reliability, and robustness of the prediction model through appropriate performance metrics and validation techniques.
3. To develop a user-friendly, web-based application that visualizes predictions in an interactive format and features a blog page where farmers, agencies, and other users can post and read locust outbreak warnings.

1.4 Research Questions

1. How can machine learning algorithms be effectively utilized to predict locust outbreaks using publicly available environmental and agricultural datasets?
2. How can the accuracy, reliability, and robustness of the locust outbreak prediction model be evaluated through appropriate performance metrics and validation techniques?
3. How can a user-friendly, web-based application be developed to visualize locust outbreak predictions and enable user engagement through a blog for sharing warnings?

1.5. Motivation of The Study

The motivation for this study arises from the urgent need to address the devastating impact of locust outbreaks on agriculture, food security, and livelihoods, particularly in Somalia. Locust swarms have historically caused significant crop losses, leading to hunger, displacement, and economic instability. Current locust management strategies, which often rely on manual inspection and pesticide application, are resource-intensive, reactive, and insufficient to mitigate widespread damage.

The increasing availability of machine learning techniques and open-access environmental datasets provides an opportunity to develop cost-effective, predictive systems that can enable early interventions. This study is motivated by a desire to harness these technologies to create a practical and scalable solution tailored to the needs of communities with limited resources.

As students in Somalia, where locust outbreaks frequently exacerbate food insecurity, we are driven by the potential to make a real-world impact. By focusing on simplicity, affordability, and usability.

1.6 Scope of The Study

This study focuses on developing a machine learning-based system for predicting desert locust outbreaks in Somalia, utilizing publicly available.

Lightweight algorithms, including logistic regression and decision trees, will be explored due to their low computational requirements and interpretability The target audience includes local farmers, agricultural agencies, and policymakers, with an emphasis on usability and accessibility.

1.6.1 Time Scope of The Study

The time of this project will be conducted between January 2025 to August 2025

1.7 Significant of The Study

This study addresses the critical issue of desert locust outbreaks, which threaten food security and livelihoods in Somalia. By focusing on resource-constrained settings, the research provides a lightweight and cost-effective machine learning solution that can be deployed in areas with limited technological infrastructure.

The proposed system empowers local communities by offering early warnings and actionable insights, enabling proactive measures to mitigate crop losses. Additionally, the study contributes to the growing body of knowledge on locust prediction by

demonstrating how publicly available data and simple algorithms can be effectively utilized in real-world contexts. Finally, the framework promotes sustainability by reducing reliance on chemical pesticides, supporting environmentally friendly agricultural practices.

1.8 Organization of the study

Chapter I: Introduction to study –: This chapter outlines the background, problem statement, objectives, research questions, scope, and significance of the study.

Chapter II: Literature Reviews –: A review of existing research and methodologies related to locust outbreak prediction, highlighting gaps and opportunities for improvement.

Chapter III: Methodology –: This chapter describes the data collection process, machine learning techniques, model development, and evaluation methods.

Chapter IV: System Analysis and Design –: This chapter describes the system architecture and design of the proposed machine learning framework for locust prediction. It includes the selection of algorithms, data pre-processing steps, and the overall workflow of the system.

Chapter V: Implementation –: This chapter explains the practical implementation of the system, including the tools, technologies, and platforms used. It also discusses the challenges faced during implementation and how they were addressed.

Chapter VI: Discussion –: This chapter presents the findings of the study, including the performance of the machine learning models in predicting locust swarms. It interprets the results, compares them with existing studies, and discusses their implications for locust management in Somalia.

Chapter VII: Conclusion and Recommendation –: This chapter summarizes the key findings of the study, emphasizing its contributions to locust prediction and mitigation. It

provides practical recommendations for policymakers, farmers, and researchers, as well as suggestions for future studies.

CHAPTER II LITERATURE REVIEW

2.1 Introduction

A locust is a type of short-horned grasshopper that, depending on the surroundings, can vary its behaviour dramatically from living alone to forming enormous swarms. Large-scale crop and vegetation consumption by these swarms causes significant agricultural harm and food shortages. Favourable weather patterns, such more rainfall and vegetation growth, which create perfect nesting grounds, frequently cause locust outbreaks (Showler et al., 2021)

A swarm is a dense aggregation of locusts that move collectively over vast distances, sometimes spanning several countries. These swarms can travel at speeds of up to 150 km per day, consuming their own body weight in vegetation. A single swarm can devastate entire farmlands, stripping them of crops and pasture in hours, severely impacting food production and local economies (FAO, 2021)

Machine learning is a subset of artificial intelligence that enables computers to analyze data patterns and improve their performance on tasks without explicit programming. ML techniques, such as neural networks and decision trees, are widely used for predictive analytics in various fields, including weather forecasting, healthcare, and agriculture. By training on large datasets, ML models can recognize trends and make real-time predictions, enhancing decision-making in complex environments (Ian, Goodfellow et al., 2016).

Machine learning is applied in locust swarm prediction by analysing environmental variables such as precipitation, soil moisture, wind patterns, and temperature. These models process vast datasets from satellite imagery, meteorological stations, and historical swarm movements to forecast locust breeding and migration. By providing early warnings, ML-powered locust monitoring systems help farmers and authorities implement timely control measures, reducing potential damage (Meynard et al., 2020).

Prediction involves using data and statistical models to estimate future events or trends. In the context of locust management, prediction models analyse past and present environmental data to anticipate swarm formation, movement, and breeding sites. Effective predictions allow governments and farmers to prepare control strategies, such as targeted pesticide application and habitat management, minimizing agricultural losses (Latchininsky et al., 2016).

A system is a structured framework comprising multiple interconnected components working together to achieve a specific goal. In locust control, a prediction system integrates data collection (satellites, drones, weather sensors), data processing (ML algorithms, GIS mapping), and decision support tools (alerts, risk assessments) to provide timely warnings about locust activity. Such systems enhance preparedness and response efficiency in affected regions (Kimathi et al., 2020).

An outbreak refers to a sudden increase in locust populations due to favourable ecological conditions, leading to large-scale crop destruction. Outbreaks occur when prolonged rainfall, increased humidity, and abundant vegetation create optimal breeding conditions, allowing locust numbers to escalate rapidly. If left uncontrolled, outbreaks can escalate into full-scale plagues, causing widespread food shortages and economic devastation(Showler et al., 2021).

2.2 Impact of Locust Swarms

The frequency and duration of desert locust outbreaks have varied significantly over the course of several decades. There were five significant plagues that lasted up to 14 years throughout the first 60 years of the 20th century, with plagues occurring in about 80% of those years. The incidence and duration of these epidemics have drastically decreased since 1963, though, with outbreaks now happening maybe once every ten to fifteen years and rarely lasting longer than three years. The afflicted countries' management techniques have changed from a curative to a preventive strategy as a result of this shift in locust behavior. The 2003–2005 regional epidemic serves as evidence that the preventive approach is far more cost-effective, it could have paid for 170 years of preventive control in West Africa, but instead cost about $500 million to control. The extensive use of chemical pesticides and advancements in early warning and monitoring systems, which have enabled more efficient control, are two of the reasons for the decrease in desert locust plagues over the previous 50 years.(Cressman, 2016)

Locust swarms have historically posed significant threats to agriculture and economies, particularly in regions like Eastern Africa. A study by Sokame and his team highlights that desert locust infestations can cause crop production losses ranging from 42% to 69%, severely impacting food security and livelihoods. The upsurge of this devastating pest is the principal threat to food security, industrial raw materials, and export incomes in the affected areas. Locust swarms can vary in size, with each square kilometer containing at least one million locusts, leading to significant and extensive agricultural crop losses(Sokame et al., 2024).

The most direct and severe impact of locust swarms is the destruction of crops. A single swarm, comprising billions of locusts, can consume vast amounts of vegetation in a short period, leading to substantial losses in food production. According to the Food and Agriculture Organization (FAO), a single square kilometer of locusts can eat as much

food in one day as 35,000 people. This devastation has far-reaching implications, particularly for agrarian economies that rely heavily on crop yields for sustenance and trade. Countries in Africa, the Middle East, and South Asia have repeatedly faced agricultural crises due to locust invasions, with staple crops such as wheat, maize, and sorghum being the primary casualties(Anjita et al., 2024).

Locust swarms, particularly desert locusts (Schistocerca gregaria), have long posed a serious threat to agricultural production, food security, and the environment in many regions across Africa, the Middle East, and Southwest Asia. The vast expanses of land covered by desert locust swarms and their rapid reproduction cycles cause severe damage to crops, significantly affecting the livelihoods of farmers. The ability of locusts to travel great distances in search of food further exacerbates the difficulties faced by agricultural communities, as they may encounter unpredictable and large-scale infestations.

(Landmann, Tobias, 2023)

2.3 Traditional Methods

Traditional approaches, particularly the use of chemical pesticides, have been the mainstay of locust management for many years. Because of their quick success in lowering locust populations, these pesticides were thought to be the main remedy for locust epidemics. The issue of crop destruction might be resolved right away by immediately killing locusts by spraying huge regions with chemical agents like pesticides. However, there are serious health and environmental concerns associated with these procedures. The extensive use of chemical pesticides has been connected to detrimental effects on biodiversity, non-target creatures, and human health. Furthermore, the buildup of pesticide residues in the environment can contaminate soil and water, endangering ecosystems over the long run.(Finch et al., 2024).

A study from (Sharma, 2014) examines how locust control has evolved from traditional methods (like manual interventions and broad-spectrum pesticides) to modern technologies such as GPS, GIS tools, and satellite imagery. It highlights the failures of older approaches, which struggled to prevent plagues in regions like Africa and South Asia, and contrasts them with newer strategies like early warning systems and targeted pesticide spraying. The paper emphasizes how these advancements have improved response times and reduced crop losses, particularly in countries like India where locust outbreaks are frequent.

(Gebregiorgis et al., 2025)Their research identifies key weaknesses in current locust management, including poor coordination between countries, limited resources for surveillance, and reliance on reactive measures. It notes that many affected nations lack the infrastructure to detect outbreaks early, allowing swarms to spread unchecked. The study also discusses emerging challenges like climate change, which creates favorable breeding conditions for locusts, and calls for stronger international cooperation and investment in preventive technologies.

(Sarkar, 2020)Their paper focuses on the unintended consequences of chemical pesticides, which remain the primary tool for locust control. It explains how these chemicals contaminate soil, water, and non-target species, posing health risks to farmers and local communities. The study also highlights the environmental damage caused by widespread pesticide use, including biodiversity loss, and advocates for safer alternatives like biopesticides and integrated pest management to reduce harm.

2.4 Machine Learning for Locust Prediction

The model is trained using a labeled dataset in which a category is assigned to each sample. After that, the model learns to associate these categories with input features. Classification can be used to solve a number of issues, like determining if an email is spam or identifying objects in pictures. Decision trees, Support Vector Machines (SVM), and k-Nearest Neighbors (k-NN) are examples of common algorithms.(Jiang, 2021).

Another crucial component of supervised learning is regression, which aims to predict continuous values as opposed to categorical classifications. The model gains the ability to map inputs to a continuous output variable through regression exercises. It can be used, for instance, to estimate stock prices or predict home prices based on characteristics like location, size, and number of bedrooms. While polynomial regression, ridge regression, and LASSO regression are more sophisticated techniques that aid in producing more precise predictions, simple linear regression simply fits a line to the data.(Jiang, 2021).

In the machine learning paradigm known as "supervised learning," the model is trained using a dataset that includes input features together with their corresponding labels. By reducing the discrepancy between the expected and actual values, the model learns to translate the inputs into the right output. This method is quite adaptable for a variety of real-world issues because it can be applied to both classification and regression tasks. In domains where labeled data is available for training, such as medical diagnosis, speech recognition, and image classification, supervised learning is frequently used.(Jiang, 2021).

Unsupervised learning works with unlabeled data, as opposed to supervised learning. Without knowing the results beforehand, the goal is to find hidden patterns or structures in the data. Clustering, which groups data points into clusters according to similarity, and dimensionality reduction, such Principal Component Analysis (PCA), which streamlines

data while preserving significant features, are common unsupervised learning approaches. For tasks like feature extraction, anomaly detection, and customer segmentation, unsupervised learning is helpful.(Jiang, 2021).

In recent years, advancements in technology have significantly enhanced locust monitoring and management strategies. A notable development is the application of Doppler weather radars for real-time tracking of desert locust swarms. A study by (Anjita et al., 2024) demonstrated a systematic approach to distinctly identify and monitor concentrations of desert locust swarms using single and dual-polarization Doppler weather radars. This method allows for near real-time detection and tracking of locust movements, providing critical data to inform timely control measures and mitigate the impact on agriculture. The integration of radar technology into locust management frameworks represents a significant leap forward in early warning systems, enabling more precise and proactive responses to locust invasions.

The influence of climate on desert locust populations has been a subject of extensive research, with recent studies employing advanced statistical models to assess outbreak risks. For instance, a study published in Scientific Reports(Retkute et al., 2024) evaluated the suitability and likelihood of desert locust epidemics occurring in India by utilizing two widely recognized statistical models. The research highlighted the critical role of climatic factors such as temperature, precipitation, and soil moisture in determining the potential for locust outbreaks. By understanding these relationships, the study provides valuable insights into the environmental conditions that favour locust proliferation, thereby aiding in the development of predictive models and targeted intervention strategies.

The integration of remote sensing data into locust research has provided new avenues for monitoring and managing locust populations. A comprehensive review by (Barnhart et al., 2021) summarized remote sensing-based studies for locust management over the past

four decades, revealing significant progress and identifying gaps for further research. The review highlighted the use of various remote sensing technologies to monitor environmental variables such as vegetation cover, soil moisture, and climatic conditions, which are critical for predicting locust breeding sites and migration patterns. The study emphasized the potential of unmanned aerial vehicle (UAV) systems to offer more effective and rapid locust control measures, suggesting that the full capacity of available remote sensing applications has yet to be fully exploited in locust management.

Modeling locust population dynamics has become an essential tool in predicting and managing locust outbreaks. A study by (Word Ries et al., 2024) developed an integrated modeling framework designed to predict the behavior of migratory locust populations. This framework combines ecological and climatic data to simulate locust population dynamics and migration patterns, providing a valuable tool for decision-makers in implementing timely and effective control measures. The study underscores the importance of such models in understanding the complex interactions between locust populations and their environment, ultimately contributing to more effective locust management strategies.

The use of agent-based and continuous models has provided new insights into locust behavior and swarm dynamics. (Bernoff et al., 2020) developed models to study hopper bands of the Australian plague locust, focusing on how resource consumption mediates pulse formation and geometry. The research demonstrated that resource-dependent behavior plays a crucial role in the formation and movement of locust bands. These findings suggest that incorporating feeding behaviors into modeling efforts can enhance the understanding of locust dynamics and inform more effective control strategies.

2.5 Mitigation Strategies Using AI

Artificial Intelligence (AI) has emerged as a transformative tool in pest management, offering innovative solutions to predict, monitor, and control pest populations with unprecedented precision. By analyzing vast datasets encompassing environmental conditions, pest behaviors, and historical infestation patterns, AI enables the development of predictive models that forecast pest outbreaks. For instance, in West Africa's Sahel region, data scientists have utilized machine learning algorithms to anticipate locust swarms, thereby facilitating timely interventions to prevent devastating agricultural losses(Wight, 2024).

Incorporating AI into Integrated Pest Management (IPM) systems enhances the effectiveness of pest control strategies. AI-powered drones and sensors facilitate continuous monitoring of pest populations, allowing for early detection of infestations. By distinguishing between healthy and infested plants, these technologies enable farmers to address issues promptly, minimizing crop damage and reducing reliance on chemical pesticides. This approach not only preserves crop yields but also promotes environmental sustainability by decreasing the ecological footprint of agricultural practices(Hopkins, 2024).

The automation of pest detection through AI has revolutionized real-time monitoring capabilities. Advanced image recognition systems, employing deep learning algorithms, can identify specific pest species and assess infestation levels. For example, AI-equipped drones capture high-resolution images of crops, which are then analyzed to detect pest presence and density. This real-time data collection allows for immediate response measures, such as targeted pesticide application, thereby enhancing the precision and efficiency of pest control operations(Pramod et al., 2020).

Predictive analytics, powered by AI, offer proactive pest management solutions by forecasting potential infestation hotspots. By analyzing historical data alongside current environmental variables, AI models can predict where and when pest outbreaks are likely to occur. This foresight enables farmers and agricultural stakeholders to implement preventive measures in anticipation of pest emergence, thereby safeguarding crops and reducing economic losses. Such predictive capabilities are instrumental in developing strategic, data-driven approaches to pest management(shannon@smcnamara.com, 2024).

2.6 Challenges and Opportunities

Integrating remote sensing and machine learning for regional-scale habitat mapping presents several challenges. One significant issue is the complexity of accurately estimating environmental variables, such as rainfall, in arid and semi-arid regions where desert locusts thrive. Remote sensing techniques often struggle with low detection probabilities and high overestimation rates in these environments, leading to potential inaccuracies in habitat predictions. For instance, rainfall detection probabilities in these regions can range from 70% to as low as 20%, complicating the modeling process. Despite these challenges, the integration of remote sensing and machine learning offers substantial opportunities for improving desert locust monitoring and management. Advancements in machine learning algorithms, such as deep neural networks, have demonstrated high accuracy in predicting locust habitats. A study evaluating six machine learning algorithms in Niger and Sudan found that k-nearest neighbor and deep neural networks achieved accuracy scores of 88%, indicating their potential for effective habitat mapping. (Rhodes & Sagan, 2022).

Looking ahead, there is significant potential to enhance desert locust monitoring through the integration of emerging technologies. The fusion of multi-source remote sensing data with advanced machine learning techniques can lead to more accurate and timely predictions of locust breeding grounds. This approach not only improves early warning

systems but also aids in the efficient allocation of resources for locust control, ultimately contributing to better food security in affected regions(Klein et al., 2021).

Locust swarms exhibit highly complex collective behavior, making their study and control challenging. One significant challenge is their unpredictable movement patterns, influenced by environmental conditions such as wind, temperature, and food availability. This unpredictability makes forecasting their trajectory difficult, complicating mitigation efforts. Another challenge is density-dependent phase polyphenism, where locusts transition from solitary to gregarious forms, altering their behavior, physiology, and movement dynamics. Managing this shift is difficult, as gregarious locusts form massive swarms that travel long distances, consuming vast amounts of crops. Resource competition within swarms is another issue, as individuals must balance movement coordination with food availability, leading to uneven swarm distributions and internal conflicts.(Goswami et al., 2024).

Locust swarms present numerous challenges that significantly impact agriculture, food security, and livelihoods, particularly in regions like Eastern Africa. One of the primary challenges is early detection and monitoring, as locust breeding grounds are often in remote, inaccessible areas, making surveillance difficult. Climate variability further exacerbates the issue, as changes in temperature, rainfall, and wind patterns influence locust population dynamics and movement unpredictably. Rapid reproduction and mobility make control measures difficult, as swarms can travel hundreds of kilometers in a single day, quickly overwhelming response efforts.(Sokame et al., 2024)

2.7 Related Works

(Abraham Kuriakose et al., 2023) developed an innovative system that integrates artificial intelligence with unmanned aerial vehicles (drones) for real-time locust management. Their work details how drones equipped with high-resolution cameras capture aerial images, which are then processed by convolutional neural networks (CNNs) to detect clusters of locusts. The system follows a comprehensive workflow that includes image capture, data pre-processing, feature extraction, and automated decision-making to activate targeted pesticide spraying. This research emphasizes the significant reduction in response time and improved efficiency in controlling locust outbreaks, ultimately aiming to minimize crop damage and support sustainable agricultural practices.

(Sidra Khan et al., 2024) introduced LocustLens—a system that fuses satellite-derived environmental data with historical locust occurrence records to predict potential desert locust breeding sites. The research describes a multi-stage process where environmental variables such as soil moisture, temperature, and precipitation are extracted from remote sensing sources and combined with locust data. Multiple machine learning models, including deep learning techniques, are then applied to uncover the complex relationships between these factors and locust breeding behaviour. The study provides a thorough explanation of data collection, pre-processing, and model evaluation steps, demonstrating that a data fusion approach can significantly enhance early-warning capabilities against locust outbreaks.

(Yusuf et al., 2024) proposed a geospatial framework to forecast locust breeding grounds by combining extensive locust observation records from the UN-FAO with high-resolution multispectral satellite imagery. Their approach involves detailed mapping of environmental factors—such as vegetation cover and soil properties—over large areas, followed by training deep learning models to recognize patterns associated with locust breeding. The paper carefully explains each stage from data curation and cleaning to

model design and validation, and it shows that a Prithvi-based model (fine-tuned with NASA's Harmonized Landsat and Sentinel-2 data) can accurately identify high-risk areas. This research is vital for providing actionable early warnings that help authorities deploy timely preventive measures.

In their 2021 study, (Yusuf et al., 2022) addressed a major limitation in locust data—the presence-only nature of most records—by developing advanced methods to generate pseudo-absence points. The researchers compared several techniques, such as random sampling and environmental profiling, to ensure that the generated absence data were ecologically realistic. By incorporating these enhanced datasets into a logistic regression model, the study achieved significantly improved predictions of locust breeding grounds. The paper describes the entire process in detail, from data augmentation and feature selection to model training and performance evaluation, illustrating how even simple models can be highly effective when supported by quality data enrichment.

(Klein et al., 2021) their research provided a comprehensive analysis of remote sensing applications in locust management over the past four decades. It highlights the significant role of remote sensing in monitoring locust habitats, particularly through the use of optical sensors to derive vegetation indices and land cover maps. The review identifies a strong focus on species such as the desert locust, migratory locust, and Australian plague locust, while noting a scarcity of studies on other destructive species. The authors emphasize the underutilization of available remote sensing resources and advocate for future research to explore the full potential of these technologies in locust management.

(Gómez et al., 2019) explored the application of machine learning methods to identify potential desert locust breeding areas. By incorporating environmental predictors such as surface temperature, NDVI, and soil moisture at root zone, the study achieves high accuracy in detecting locust presence. The findings underscore the potential of integrating

Earth observation data with advanced modeling techniques to enhance early warning systems and preventive measures against locust outbreaks.

(Ellenburg et al., 2021) their research evaluates the influence of soil physical characteristics, particularly texture and moisture conditions, on desert locust breeding. Utilizing model-derived soil moisture estimates and known locust observations, the research identifies optimal thresholds for breeding conditions. The results demonstrate that combining soil texture with modeled soil moisture content can effectively predict locust egg-laying periods, contributing valuable insights for monitoring and preventive strategies.

(Chen et al., 2020) their research employs a logistic regression model to simulate potential habitats of desert locusts across multiple continents. By analyzing environmental data from various remote-sensing sources, the study identifies key factors influencing locust distribution, such as temperature and leaf area index. The model predicts a periodic movement pattern of potential habitats and highlights regions that should prioritize monitoring and rapid response efforts to mitigate the impact of locust infestations.

(Samil et al., 2021) focused on forecasting the regional distribution of locust swarms by employing recurrent neural networks (RNNs). Their approach uses time-series data from historical swarm records and environmental variables like soil moisture and vegetation indices to capture the sequential nature of locust outbreaks. The study provides a clear explanation of the data pre-processing, model architecture, and training process, showing that RNNs can effectively learn the temporal dependencies that underlie swarm formation. The model's predictions offer valuable insights into potential outbreak regions, thereby supporting proactive measures for locust control.

(Sun et al., 2022) proposed a dynamic forecasting model that combines a support vector machine (SVM) with a multivariate time lag sliding window technique. The model is designed to capture the delayed impact of environmental changes—such as rainfall, soil

moisture, and vegetation dynamics—on locust swarm emergence. The research explains in depth how the sliding window method selects the most informative historical time segments for prediction and how these are incorporated into the SVM. By improving both accuracy and lead time, this model offers a robust tool for early locust detection and timely implementation of control measures, ultimately aiming to reduce crop losses.

In another contribution, (Yusuf et al., 2024) developed an early detection system that integrates machine learning with geospatial analysis to forecast locust breeding sites. Their work involves enriching locust presence data with carefully generated pseudo-absence points using environmental profiling techniques, and then applying logistic regression to predict outbreak-prone areas. The system is described step by step—from data collection and enrichment to model calibration and validation—highlighting its potential to provide early alerts that can help mobilize preventive measures quickly. This research demonstrates the power of combining advanced data generation methods with simple yet effective machine learning models to enhance locust management across Africa.

(Abraham Kuriakose et al., 2023) proposed a swarm drone system that employs the YOLOv8 algorithm for real-time locust detection. In this system, a fleet of drones captures live video data from the field and processes it on board using a state-of-the-art CNN model to accurately detect locust clusters. The study describes how the detected locust locations are communicated to a central control system, which then directs the drones to perform synchronized pesticide spraying with the help of the ArduPilot control system. This detailed integration of object detection, real-time data processing, and coordinated drone action highlights a promising technological solution that can rapidly address locust infestations, reducing both crop loss and environmental impact.

Recent studies have increasingly focused on integrating remote sensing data with machine learning techniques to enhance habitat mapping for desert locust monitoring.

One such study evaluated six machine learning algorithms across regions in Niger and Sudan, finding that k-nearest neighbor (kNN) and deep neural networks (DNN) performed best, each achieving an average accuracy of 88% and F-1 scores around 89%. The research underscores the potential of deep learning models in developing transferable representations for sustainable locust management strategies(Rhodes & Sagan, 2022).

Climate change poses significant challenges to pest management, as shifting climatic conditions can alter species distributions. A study utilizing the MaxEnt model predicted the current and future potential distributions of the desert locust, Schistocerca gregaria, under different climate scenarios for 2050 and 2070. Using 226 occurrence records and nine bioclimatic variables, the model demonstrated high precision, with mean AUC values ranging from 0.929 to 0.940. Notably, bioclimatic variables Bio11 (mean temperature of the coldest quarter) and Bio18 (precipitation of the warmest quarter) contributed significantly to the model, accounting for 51.4% and 17.3% respectively. The findings suggest a potential decrease in high-suitability habitats for desert locusts in the future, emphasizing the need for adaptive management strategies(Geng et al., 2020).

The 2020 desert locust plague severely impacted food security across Africa and Asia, prompting research into predictive monitoring methods. One study employed a Hidden Markov Model (HMM) to predict locust plague severity in croplands using time-series data from multiple remote sensing sources. Evaluations against ground truth data yielded overall accuracies of 0.78, 0.71, 0.74, and 0.72 for predictions in April through July, respectively. Additionally, change detection methods applied to hyperspectral images allowed for quantitative assessment of locust-induced damage to croplands, demonstrating the effectiveness of HMM-based approaches in forecasting plagues and evaluating their impacts(Shao et al., 2021).

A novel multi-scale methodology has been proposed to model habitat suitability for various locust species by integrating ecological niche modeling with high-resolution

remote sensing data. This approach was applied to the Italian locust in Northern Kazakhstan, the Moroccan locust in Southern Kazakhstan, and the desert locust in regions including Ethiopia, Djibouti, and Somalia. By incorporating climatic variables such as temperature and rainfall, along with Sentinel-2 time-series data to assess current vegetation states and land use, the fused Habitat Suitability Index (HSI) models achieved Area Under Curve (AUC) performances of 0.747 for the Italian locust, 0.850 for the Moroccan locust, and 0.801 for the desert locust. This multi-scale approach provides detailed, up-to-date spatial information on breeding suitability, facilitating prioritized risk assessments and early interventions in locust pest management(Klein et al., 2022).

(Gómez et al., 2018) applied machine learning to remotely sensed soil moisture data in order to monitor and predict locust breeding areas. Their approach uses random forest classifiers to analyse variations in soil moisture—a key factor in locust breeding—and links these changes to locust occurrence records. The study provides a comprehensive description of how satellite data from ESA's Climate Change Initiative is pre-processed and used to extract relevant environmental features. By demonstrating that these remote sensing indicators can reliably predict locust swarm conditions, the work lays an important foundation for developing AI-based monitoring systems that enable timely preventive actions.

(Ye et al., 2020) tackled the challenge of accurately identifying different locust species through a convolutional neural network (CNN) designed to classify RGB images of locusts. Their research outlines a detailed methodology that includes collecting diverse image datasets, pre-processing the images, and training the CNN with advanced techniques such as batch normalization to enhance stability and accuracy. The system aims to provide rapid, automated species identification, which is critical for selecting appropriate control measures. This study is particularly valuable because it demonstrates

how AI can support field experts and farmers in making informed decisions by quickly determining the exact locust species present during an outbreak.

(Landmann, Tobias, 2023) proposed an early-warning system that integrates fuzzy logic with environmental monitoring to predict the hatching and subsequent swarm formation of desert locusts. By designing membership functions and rule-based systems, the model effectively handles the uncertainty and imprecision of environmental data such as rainfall, temperature, and soil moisture. The study explains each step—from defining the fuzzy sets to combining the inputs into a coherent prediction—and demonstrates that this approach can forecast locust emergence with around 82% accuracy. The work emphasizes the importance of accommodating real-world data variability, ultimately providing a flexible tool for timely locust control interventions.

(Maryam Tabar et al., 2021) introduced the PLAN (Predictor of Locust Activity and movement) model, which uses deep learning to forecast the movement of locust swarms. Their framework integrates multiple data sources including crowdsourced locust sightings and remote-sensed environmental variables by combining convolutional layers for spatial feature extraction with LSTM units for temporal analysis. The paper explains in detail the process of data fusion, model training, and hyperparameter tuning, and it shows that the PLAN model achieves high accuracy in predicting both locust presence and migration patterns. This comprehensive approach provides decision-makers with actionable predict, helping to plan targeted interventions that mitigate the adverse effects of locust invasions on agriculture.

2.8 Gaps & Contributions

1. Unlike previous studies, which did not provide a user-friendly interface for end users, we developed an intuitive user interface designed specifically for farmers and agricultural decision-makers to easily interact with the system.

2. While earlier research efforts trained a maximum of seven models, our study expanded this by training thirteen models, each demonstrating superior accuracy and performance.

2.9 Proposed System

The proposed system aims to improve locust management through a machine learning-based Early Warning and Prediction System for Locust Outbreaks. It uses publicly available environmental and agricultural data to accurately forecast locust outbreaks, addressing the shortcomings of traditional methods like manual surveys and pesticide spraying.

The system combines various machine learning algorithms, including SVM, LightGBM, and Random Forest, to analyze key factors such as Perticipitation, temperature, and soil moisture. By using a dataset from the Food and Agriculture Organization (FAO), it identifies patterns for near-real-time predictions, which are vital for timely interventions. The user-friendly web interface allows farmers and policymakers to easily input data and receive predictions. The backend, built with Python's Flask framework, processes data efficiently. Additionally, an analytics dashboard visualizes trends, helping users understand locust behavior.

In summary, this system provides a proactive approach to locust management, enhancing prediction accuracy and supporting sustainable farming practices. It has the potential to significantly boost food security and economic stability in regions affected by locust outbreaks.

CHAPTER III: METHODOLOGY

3.1 Introduction

In areas like Somalia, locust infestations are becoming more frequent and severe, which has put livelihoods, food security, and agricultural output at serious risk. The quick spread and unpredictable nature of locust swarms have made traditional tracking and management techniques ineffective. In order to predict locust behavior and movement patterns based on a variety of environmental factors, such as Maximum Temperature, Soil Moisture, and Precipitation. The main goal of this research is to use sophisticated machine learning techniques, such as supervised learning algorithms. for efficient reaction plans and early warning systems. The procedures used to create the locust swarm prediction system, including data collection, preprocessing, model selection, training, and implementation.

It also provides a thorough description of how machine learning might be used to enhance locust swarm prediction and mitigation efforts in Somalia by outlining the system's architecture, development environment, and hardware and software requirements. In addition to improving prediction accuracy, this strategy seeks to develop a long-term solution for controlling locust swarms in a setting with limited resources.

3.2 System Description

In order to lessen the catastrophic impact that locust swarms have on agriculture, the method created in this work incorporates machine learning algorithms to predict locust swarm occurrences in Somalia. The system integrates information from dataset sources (e.g., Region, Country-name, start-year, start-month, Max temperature, Soil Moisture, Precipitation, and locust-present). To predict locust behaviour, this extensive data is incorporated into machine learning models, mostly Logistic Regression, Decision Trees, k-Nearest Neighbors, Support Vector Machines and Random Forest. These models are trained to identify environmental variable patterns that point to swarm formation and

locust movement. By doing this, the system can predict the probability of locust swarms in particular places, giving impacted areas early warnings.

Following projection, the system provides farmers, stakeholders, and local government with actionable information and warnings that enable prompt actions. This could entail putting mitigating techniques into practice, such as localized evacuations, pesticide spraying, or rerouting the movement of impacted populations. The system can process massive amounts of real-time data and deliver predictions quickly since it is designed to be flexible and expandable. Additionally, all stakeholders, including government organizations and agricultural specialists, may easily access and comprehend predictions thanks to the user-friendly interface. The technology greatly improves the ability to control locust infestations and lessens their negative effects on Somalia's agriculture sector by offering preemptive prediction

3.3 System Architecture

The Locust Prediction System is designed using a structured architecture that integrates multiple components to ensure efficient data processing, model execution, and user interaction. The architecture follows a modular approach, where data collection, preprocessing, model prediction, and result visualization are managed independently yet interconnectedly. This ensures scalability, maintainability, and flexibility in integrating new features or improving existing functionalities.

The following figure illustrates the overall system architecture, depicting how different components interact within the system to facilitate locust prediction and decision-making.

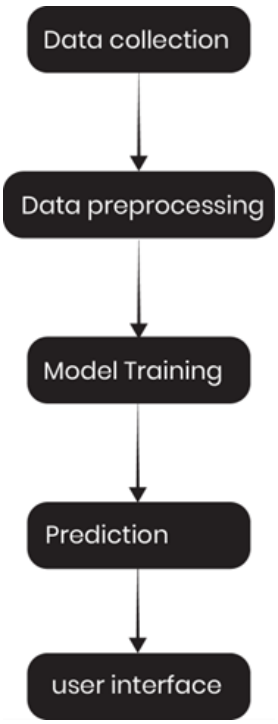


Figure 3. 1 System Architecture

3.4 System Features

The Locust Prediction System is designed with a set of key features that enhance user experience and provide valuable insights through data-driven analysis. These features aim to make the system user-friendly, informative, and effective in delivering locust predictions. The system includes interactive data visualizations, predictive analytics, and essential functionalities to ensure accessibility and ease of use. Each feature plays a crucial role in improving the usability of the system, enabling users to access predict, reports, and other relevant information efficiently. The following are the core features of the system.

3.4.1 Authentication and Authorization

The system uses a secure login and registration process. After logging in, users receive a secure token (JWT) that ensures they can only access their own private data.

3.4.2 Prediction Generation

Users can get instant locust outbreak forecasts by entering environmental data. The system provides a simple "YES" or "NO" prediction, along with a confidence score showing the probability.

3.4.3 Analytics Dashboard

The analytics dashboard uses charts and graphs to visualize a user's prediction history. This makes it easy to spot outbreak trends, view risk patterns by location, and understand key environmental factors.

3.4.4 Prediction Feedback

Users can submit feedback on whether a prediction was accurate in the real world. This information is used by the system to improve the forecasting model over time.

4.4.5 Blog and Content Management

The platform includes a blog where users can create, edit, and delete posts. This allows them to easily share important news and to warn the community from a locust outbreak possibility.

4.5.6 Reports and Data Export

Users can view a detailed history of all their past predictions in a report. This report can be exported for offline use as a CSV or PDF file, or sent directly to a printer.

4.5.6 Profile and Account Management

Users can manage their profile information and change their password. A "Danger Zone" also allows for irreversible actions like a Factory Reset to clear all data or a full Account Deletion.

3.5 System Methodology

This section outlines the methodology followed in developing the Locust Prediction System, detailing the process from data collection and preparation to model selection, training, and final implementation. The system leverages machine learning techniques to analyze environmental factors contributing to locust outbreaks. It follows a structured workflow that ensures the dataset is properly processed and utilized for accurate locust presence prediction.

3.5.1 Dataset

The Locust Prediction System is designed to analyze environmental factors that contribute to locust outbreaks, enabling early warnings. The system utilizes a dataset sourced from the Food and Agriculture Organization (FAO), which contains historical records of locust presence alongside key environmental variables. The dataset consists of **79,529** records and **8 columns**, each representing a crucial factor in understanding locust breeding and migration patterns.

To enhance predictive accuracy, the dataset includes climate-related attributes such as precipitation, temperature, and soil moisture, which are significant indicators of locust outbreaks. The data is processed to extract meaningful patterns, allowing the system to provide reliable prediction for affected regions.

By leveraging this dataset, the system can assess how environmental conditions contribute to locust population growth, allowing for proactive mitigation measures.

3.5.2 Data Preparation

Before training the machine learning models, the dataset undergoes several preprocessing steps to ensure quality and consistency.

The main steps include:

- **Handling Missing Data:** Missing values in precipitation, temperature, or soil moisture fields can affect model performance. If missing data is minimal, the records may be removed; otherwise, missing values are imputed using the mean or median of the respective feature.
- **Encoding Categorical Variables:** The REGION and COUNTRYNAME columns contain categorical data. These are transformed into numerical representations using one-hot encoding to allow machine learning models to process them effectively.
- **Feature Scaling:** Since temperature, precipitation, and soil moisture have different numerical ranges, min-max scaling is applied to normalize the data, ensuring that no feature dominates the learning process.
- **Handling Imbalanced Data:** If the number of records where locusts are present is significantly lower than records where locusts are absent, techniques such as SMOTE (Synthetic Minority Over-sampling Technique) or random undersampling are applied to balance the dataset.
- **Outlier Detection:** Extreme values in temperature or precipitation can distort predictions. Z-score analysis or interquartile range (IQR) filtering is applied to detect and handle outliers.

By ensuring the dataset is clean and well-processed, the Locust Prediction System improves its predictive capabilities, leading to more reliable locust outbreak forecasting.

3.5.3 Model Selection

For the Locust Prediction System, the model selection process involves evaluating multiple machine learning algorithms to determine which is best suited for accurately predicting locust outbreaks based on environmental and historical data. Selecting the appropriate models ensures reliable predict, enabling authorities and farmers to take proactive measures. The chosen models are assessed based on their ability to handle large datasets, predictive accuracy, and generalization capabilities. The selected models include:

1. Decision Tree Classification Model: A Decision Tree model is a widely used algorithm for classification tasks, making decisions by splitting data into subsets based on feature values. Each internal node represents a decision based on an attribute, while leaf nodes indicate the prediction outcome.

In the context of locust prediction, the Decision Tree model helps in classifying regions into risk levels (e.g., high, medium, or low locust infestation) based on factors such as temperature, humidity, wind speed, and vegetation index. Its interpretability makes it an effective tool for understanding how different environmental factors contribute to locust outbreaks.

2. Random Forest Classification Model: A Random Forest model is an ensemble learning method that enhances the accuracy of Decision Trees by constructing multiple trees and aggregating their predictions. It reduces overfitting and improves prediction stability.

For locust prediction, the Random Forest model analyzes multiple environmental variables simultaneously, ensuring a robust classification of infestation risks. By averaging predictions from multiple trees, it provides a more reliable and generalized forecast, helping decision-makers take necessary preventive actions.

3. K-Nearest Neighbors (K-NN) Classification Model: The K-NN algorithm is a non-parametric model that classifies new data points based on the 'k' closest observations in the dataset. It works well when patterns in historical data can be leveraged for future predictions.

In locust forecasting, K-NN identifies patterns in past locust movements and environmental conditions to classify new locations with a similar risk profile. By comparing a region's climate and vegetation with previously affected areas, K-NN helps predict potential outbreaks in similar regions.

4. Support Vector Machine (SVM) Classification Model: SVM is a powerful supervised learning algorithm that finds the optimal decision boundary between classes by maximizing the margin between different categories.

When applied to locust prediction, SVM separates affected and non-affected regions using environmental feature vectors. By identifying clear boundaries between infestation-prone and safe areas, SVM ensures precise classification, aiding in focused intervention strategies.

5. Logistic Regression Model: Logistic Regression is used for binary classification tasks, predicting the probability of an event occurring based on input features.

In this case, Logistic Regression helps determine whether a region is at risk of a locust outbreak (Yes/No) based on climatic and vegetation data. It provides a straightforward and interpretable model for rapid decision-making, allowing authorities to act quickly in response to high-risk predictions.

Each of these models is evaluated based on their precision, recall, and overall accuracy in predicting locust infestations. By using a combination of these approaches, the system ensures a robust, data-driven method for forecasting and mitigating locust outbreaks effectively.

6. Bagging Classifier: The Bagging Classifier is an ensemble method that improves stability and accuracy by training multiple base models (typically Decision Trees) on random subsets of the data and aggregating their predictions.

For locust prediction, Bagging reduces variance and enhances generalization by combining predictions from multiple classifiers. This approach minimizes overfitting to specific environmental patterns, providing more reliable risk assessments across diverse regions.

7. Extra Trees Classifier: Extra Trees (Extremely Randomized Trees) is a variant of Random Forest that introduces additional randomness by selecting random split points for features.

In locust forecasting, Extra Trees improves computational efficiency while maintaining robust performance. Its randomized splitting helps capture non-linear relationships between environmental factors (e.g., sudden vegetation changes) and outbreak likelihood.

8. AdaBoost Classifier: AdaBoost (Adaptive Boosting) iteratively improves predictions by focusing on misclassified samples from previous iterations, assigning them higher weights.

For locust risk classification, AdaBoost excels at identifying subtle environmental thresholds (e.g., soil moisture levels) that trigger outbreaks. Its adaptive nature makes it particularly effective for imbalanced datasets.

9. XGBoost Classifier: XGBoost (eXtreme Gradient Boosting) is an optimized gradient-boosting framework that combines weak learners sequentially to correct errors.

In this system, XGBoost processes complex interactions between climate variables (temperature, rainfall) and vegetation indices to predict outbreaks with high precision. Its regularization prevents overfitting to noisy field data.

10. Naïve Bayes Classifier: Naïve Bayes applies Bayes' theorem with strong feature independence assumptions, making it computationally lightweight.

For rapid locust risk screening, Naïve Bayes provides probabilistic estimates of outbreak likelihood based on historical climate-outbreak correlations. While simple, it serves as a useful baseline for more complex models.

11. LightGBM Classifier: LightGBM (Light Gradient Boosting Machine) uses histogram-based algorithms to accelerate training while handling large-scale environmental datasets efficiently.

In deployment, LightGBM's ability to process high-dimensional features (e.g., satellite-derived vegetation time series) enables real-time risk mapping with minimal computational overhead.

12. CatBoost Classifier

CatBoost handles categorical features natively and employs ordered boosting to reduce prediction bias.

For locust prediction, CatBoost automatically manages region-based categorical variables (e.g., soil type classifications) without manual encoding, improving workflow efficiency.

13. Gradient Boosting Classifier: Gradient Boosting builds an additive model stage-wise, optimizing for differentiable loss functions.

This model captures progressive environmental degradation patterns (e.g., vegetation stress accumulation) that precede outbreaks, offering early warning capabilities.

3.5.4 Training

The training process for the Locust Prediction System involves preparing the selected machine learning models using the dataset to accurately predict the presence of locust outbreaks based on environmental conditions. Given the models outlined earlier (Decision Tree, Random Forest, and K-Nearest Neighbors), the training process follows these key steps:

Training Process Overview:

- **Splitting the Dataset:** The dataset is divided into training and testing sets to evaluate model performance on unseen data. A typical split is 80% for training and 20% for testing, but these proportions can vary depending on dataset size and specific requirements.
- **Feature Selection:** Before training, relevant features are identified to ensure the model focuses on key indicators of locust outbreaks. This includes factors such as precipitation, temperature, and soil moisture—variables that have a direct impact on locust breeding and migration patterns. Feature selection is guided by domain knowledge, statistical analysis, and feature importance scores derived from initial model tests.
- **Model Training:** The selected machine learning algorithms are trained using the preprocessed dataset. Each model learns patterns from the input features and optimizes its predictions based on historical locust presence data. Hyperparameter tuning is performed to enhance accuracy and prevent overfitting, ensuring that the models generalize well to new data.
- **Model Evaluation:** After training, the models are evaluated using the test set, where metrics such as accuracy, precision, recall, and F1-score are calculated. This step helps in selecting the best-performing model for deployment. If necessary,

adjustments are made to improve performance, such as feature engineering, parameter tuning, or using ensemble methods for better predictions.

3.5.5 Implementation

The implementation of the Locust Prediction System involves integrating the trained machine learning models into a fully functional application that allows users to analyze locust outbreak risks based on environmental factors. The system is divided into two primary components: the frontend and the backend.

3.5.6.1 Frontend

The frontend serves as the user interface where users interact with the system. It is developed using modern web technologies to ensure a responsive and user-friendly experience.

Key features of the frontend include:

- **Data Input Form** – Users can input environmental variables such as precipitation, temperature, and soil moisture.
- **Prediction Results Display** – The system processes the input data and presents visualized predictions of locust presence, including risk levels.
- **Interactive Data Visualization** – Graphs and charts help users interpret historical trends and predictions, making it easier to assess locust outbreak risks.

The frontend is developed using HTML, CSS, JavaScript, and Bootstrap, ensuring cross-device compatibility.

3.5.6.2 Backend

The backend is responsible for data processing, model execution, and response handling. It is built using Flask (Python), which serves as the core framework for managing API requests and executing machine learning models.

3.6 System Development Environment

In this study, we need to establish a development environment that facilitates the implementation of the Locust Prediction System. This environment consists of a set of tools, frameworks, and programming utilities that streamline the development, testing, and deployment processes. It provides a structured workflow, enabling efficient code writing, debugging, and integration of machine learning models within a functional system.

The Development Environment includes:

- Python Flask
- Jupyter Notebook
- VS Code
- MySQL

Flask is a lightweight web framework for Python, widely used for building web applications and APIs. It follows the WSGI (Web Server Gateway Interface) standard, making it an efficient tool for handling HTTP requests and serving machine learning models as web-based services. In this project, Flask serves as the backend framework, managing API endpoints that process input data and return locust outbreak predictions.

Jupyter Notebook is an interactive development environment that allows for real-time coding, visualization, and debugging of Python scripts. It is particularly useful for data exploration, preprocessing, and model training, providing a structured way to analyze locust-related environmental data before integrating the trained models into the Flask application.

Visual Studio Code (VS Code) is a lightweight, feature-rich IDE that supports Python development. It offers built-in debugging, version control integration, and extensions that enhance code writing, collaboration, and testing. In this project, VS Code is used for

developing the Flask backend, handling API requests, and integrating the machine learning models into a deployable system.

MySQL is an open-source relational database system used to store, manage, and retrieve structured data. In research, it supports efficient handling of large datasets, making it ideal for projects like locust swarm prediction, where fast access to environmental data is essential for machine learning and real-time analysis.

3.7 System Requirements

This section details the hardware and software components needed to develop, run, and maintain the Locust Prediction System. The requirements are presented in two tables: one for hardware and one for software, ensuring that the system is both efficient and feasible within a resource-constrained environment.

3.7.1 Hardware Requirements

To achieve efficient performance, the system requires a stable hardware environment. Below are the necessary hardware components along with their specifications and purposes:

Table 3. 1 Hardware Requirements

Components	Specification	Purpose
CPU	Intel Core i3 (minimum) / Intel Core i5+ (recommended)	To handle data processing and model inference without delays.
Memory (RAM)	8 GB or higher	Ensures smooth operation during data preprocessing and model training.
Storage	128 GB HDD (minimum) / 512 GB SSD (recommended)	To store datasets, preprocessed data, and model files.
Network Interface	High-speed broadband	Required for accessing online datasets and updates if needed.

3.7.2 Software Requirements

The software environment consists of essential tools that facilitate system development, deployment, and maintenance. Below are the required software tools along with their respective versions and purposes:

Table 3. 2 Software Requirements

Software/Tool	Version	Purpose
Operating System	Windows 10+	Provides the foundational platform for system execution.
Python (Flask)	Version 3.9 or later	Web framework for processing model's data
MySql	Latest stable version	Database management for storing processed data

Table 3. 3 Required Libraries

Library	Purpose
NumPy	Supports numerical computations and array manipulations.
Pandas	Provides data manipulation and analysis functionalities.
Scikit-learn	Implements machine learning algorithms for locust prediction.
Matplotlib	Generates graphical visualizations for data representation.
Seaborn	Enhances statistical data visualization capabilities.
Joblib	Library for efficient persistence of Python objects

Table 3. 4 Integrated Development Environment (IDE)

IDE	Description
Jupiter Notebook	Interactive coding environment for Python
VS Code	Lightweight, flexible IDE with support for multiple programming languages and extensions.

CHAPTER IV: ANALYSIS AND DESIGN

4.1 Introduction

This chapter presents the detailed analysis and design of the locust prediction system we are developing. It begins by analysing the current problem and identifying the need for a more reliable solution. The chapter then introduces the system we propose and explains how it addresses the limitations of existing approaches. We will describe both the functional and non-functional requirements, conduct a feasibility study, and present the overall design of the system. This includes how data flows through the system, what kind of data we are working with, and how the prediction process works. Our goal is to make the design simple to understand while covering all important technical aspects.

4.2 System Analysis

Desert locusts are a serious threat to agriculture, especially in regions like East Africa and parts of Asia. When they swarm, they can destroy crops quickly and leave entire communities facing food shortages. While organizations and governments do try to track and respond to locust outbreaks, predicting when and where they will happen is still a major challenge.

Traditionally, locust monitoring has relied on field surveys and weather observations. However, these methods are often slow, labour-intensive, and do not make full use of the growing amounts of environmental data available today. In many cases, by the time action is taken, the locusts have already caused significant damage.

Our system aims to solve this problem by using machine learning to analyse environmental factors like rainfall, temperature, and soil moisture. By studying past locust outbreaks and the conditions that surrounded them, the model can learn to detect patterns and predict the likelihood of locust presence in the future.

This gives decision-makers and farmers a valuable tool that can help them take early action and reduce the impact of locust invasions.

This analysis phase helps us understand what the system should do, who it is for, and how it will make a difference in real-world situations.

4.3 Existing Approaches

Over the years, several methods have been used to monitor and predict locust outbreaks. Most traditional approaches rely on ground-based field surveys, satellite imagery, and expert knowledge. Organizations like the Food and Agriculture Organization (FAO) maintain locust monitoring systems where field officers report sightings, and these are analysed manually or through rule-based systems.

While these approaches have helped in early warning and response, they face several limitations:

Delay in reporting: Ground surveys take time, and by the time the data is collected and processed, the locusts may have already spread.

Limited coverage: Remote or conflict-affected areas may not be surveyed regularly.

Lack of automation: Most systems depend on manual data interpretation, which is slow and may miss subtle patterns in the data.

Poor scalability: As data volumes grow, these systems struggle to handle the complexity and quantity of information.

There have also been some attempts to apply machine learning in locust prediction, but many of them either used very limited features or did not focus specifically on the East African region, where the impact is most severe.

4.4 The Proposed System

Our proposed system introduces a machine learning-based approach to predict the presence of desert locusts. The system uses historical and environmental data such as precipitation, maximum temperature, and soil moisture to identify patterns associated with previous locust outbreaks.

The system architecture includes:

A web-based user interface where users can input current environmental conditions and receive prediction results.

A backend model trained using supervised machine learning, capable of processing the inputs and returning a prediction (presence or absence of locusts).

Target encoding and binary encoding methods applied to categorical data (e.g., country and region) and the target variable respectively to improve model accuracy.

By using this approach, we provide faster, scalable, and more accurate predictions that can support farmers, governments, and organizations in taking early and effective actions against locust outbreaks.

4.5 System Requirements

This section outlines what the system should do (functional requirements) and how it should behave (non-functional requirements).

4.5.1 Functional Requirements

Functional requirements describe what the system is expected to do and the core features that enable it to fulfil its purpose. In the case of our locust prediction system, the following components define its functionality:

User Interaction: The system provides a simple and accessible web interface where users such as agricultural officers or environmental researchers can enter relevant

environmental data including precipitation, temperature, and soil moisture. This interaction is designed to require minimal technical expertise.

Input Validation: Before the entered data is processed, it is checked to ensure it falls within a realistic and acceptable range. This step prevents the system from accepting impossible values that could negatively affect the prediction accuracy.

Data Transformation: Once validated, the input data undergoes preprocessing. Categorical fields like COCUNTRYNAME and REGION names are encoded using target encoding, while the target variable (LOCUST PRESETNT) is binary encoded. This transformation prepares the input for accurate model interpretation.

Prediction Engine: The core of the system is a trained classification model that receives the transformed input and predicts whether locusts are likely to be present or not. This decision is based on learned patterns from historical data.

Result Display: After processing, the prediction is displayed clearly to the user, highlighting whether the conditions suggest the presence of locusts. The interface emphasizes clarity and ease of interpretation, even for users without a background in data science.

Error Handling: If the input data is incomplete, improperly formatted, or outside acceptable bounds, the system provides limmediate feedback, guiding the user to correct the issue.

4.5.2 Non-Functional Requirements

Non-functional requirements define how the system performs and the qualities it should maintain to ensure a good user experience. While they don't describe specific features, they are essential for reliability, usability, and trust.

Security: The system must ensure that user inputs and any backend model processes are protected against unauthorized access or misuse. This includes using secure connections and validating data inputs to prevent vulnerabilities.

Accessibility: The application should be accessible online through any modern web browser. Users should be able to access the system anytime and anywhere as long as they have an internet connection.

Usability: A clean and intuitive interface is critical. The system should be easy to navigate, with clear instructions and responsive design so that users with varying levels of technical expertise can interact with it effortlessly.

Performance: Predictions should be generated quickly once inputs are submitted. The system should remain responsive and stable even when handling multiple requests.

Scalability: The system should be designed in a way that makes it easy to expand for example, to support more countries, additional environmental features, or future upgrades to the prediction model.

Privacy: Any data entered by users should not be stored permanently or shared with third parties. The system must ensure that personal or location-based data remains confidential.

4.6 Feasibility Study

Before developing a full locust prediction system, it's important to evaluate whether the project is practical, realistic, and worth investing time and resources into. The following aspects of feasibility were considered:

4.6.1 Technical Feasibility

The technical feasibility assesses whether the tools, technologies, and expertise needed to build the system are available. In our case, the system is developed using Python for model development and Flask for the web application. These technologies are free, open-source, and widely supported. The team has sufficient experience in machine learning, web development, and data preprocessing to implement the system effectively.

4.6.2 Economic Feasibility

The project requires minimal financial investment. Since it uses open-source software and runs on lightweight infrastructure, it can be developed and deployed without expensive hardware or software licenses. This makes the system highly affordable, especially for research purposes or public sector usage in low-resource settings.

4.6.3 Operational Feasibility

The system is intended for use by agricultural staff, researchers, or government analysts. It is designed to be user-friendly, requiring no deep technical knowledge.

4.6.4 Schedule Feasibility

The development timeline is realistic and manageable. With a clear plan for building the model, creating the interface, and testing the system, the project can be completed within a few weeks. Major components like data collection, model training, and form design have already been completed, further ensuring that deadlines can be met without delays.

4.7 System Design

This section discusses the overall design of the proposed Locust Prediction System. System design is about planning the architecture, components, data handling, and how different parts of the system interact with each other. It ensures that the system will meet its goals efficiently and reliably.

4.7.1 Data Flow Diagrams (DFD)

A Data Flow Diagram (DFD) is a graphical representation of the movement of data through a system, detailing how input is transformed by various processes into output. It serves as a tool for visualizing how data is handled within a system, making it easier to understand complex workflows by breaking them into smaller components. The DFD below represents the core flow of the Locust Prediction System.

- **User Input:** User input refers to the raw environmental data provided by users through the system interface. This includes parameters such as rainfall, soil moisture, vegetation condition, surface temperature, and region. These inputs form the foundation of the prediction process, as the accuracy of predictions depends heavily on the quality of this data.
- **Input Validation & Preprocessing:** This process validates the format and completeness of the input data, handling missing or inconsistent values. It also performs preprocessing tasks like normalization and transformation to ensure that the data is clean, standardized, and suitable for further analysis. This step is crucial for maintaining model accuracy and consistency.
- **Training Dataset:** The training dataset consists of historical locust occurrence data along with corresponding environmental features. This dataset is used during the development phase of the system to train the machine learning model, enabling it to learn relationships and patterns that indicate locust presence or absence.
- **Feature Extraction:** Feature extraction is the process of identifying and selecting the most relevant features from the input data that significantly contribute to locust prediction. These features are transformed into a format suitable for the prediction model. This step enhances the model's ability to focus on meaningful information while ignoring noise.
- **Prediction:** The prediction model is the machine learning component trained on historical data. It processes the incoming, preprocessed user input and outputs a prediction indicating whether locust presence is likely in the specified region. The model may also return a confidence score representing the certainty of its prediction.
- **Result:** The result generated by the prediction model is formatted and presented to the user. along with optional confidence metrics. The goal is to provide users with actionable insights in a clear and concise manner.

- **Logging System (Data Store):** After receiving the prediction, the user is asked whether they consent to having their input data and the prediction saved for future model improvement. If the user agrees, this data is anonymized and stored in a secure database. Logging this data enables the continuous enhancement of the system's accuracy through retraining and further analysis.

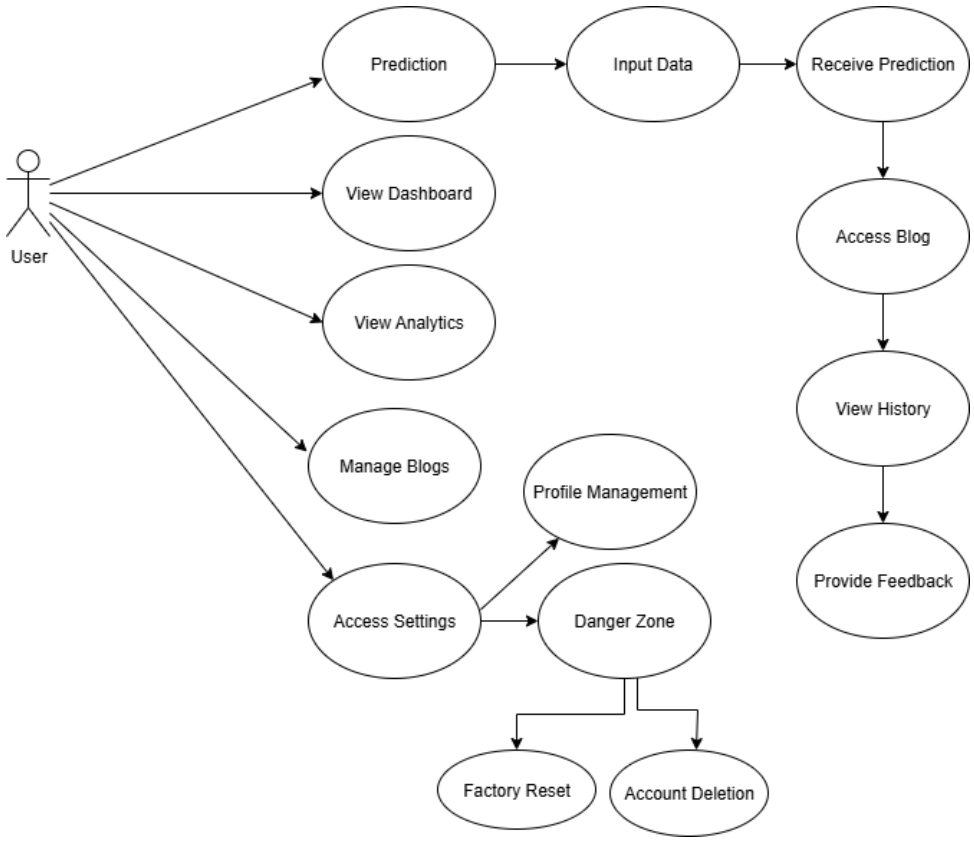
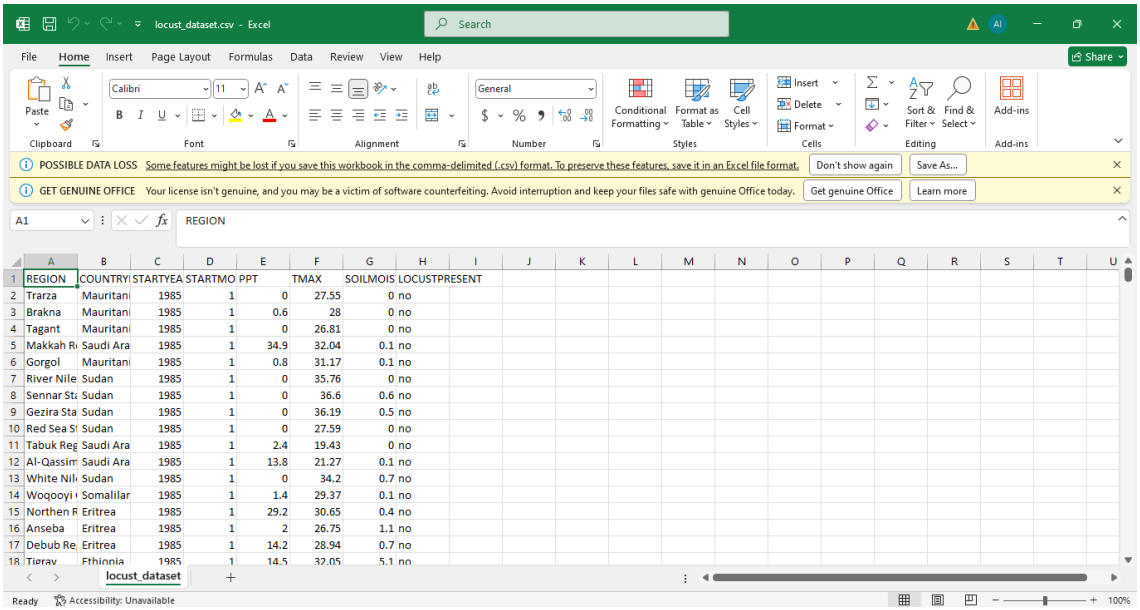


Figure 4. 1 Data Flow Diagram of LPS

4.8: Dataset Design



REGION	COUNTRY	STARTYEA	STARTMO	PPT	TMAX	SOILMOIS	LOCUSTPRESENT
Trarza	Mauritani	1985	1	0	27.55	0	no
Brakna	Mauritani	1985	1	0.6	28	0	no
Tagant	Mauritani	1985	1	0	26.81	0	no
Makkah R	Saudi Ara	1985	1	34.9	32.04	0.1	no
Gorgol	Mauritani	1985	1	0.8	31.17	0.1	no
River Nile	Sudan	1985	1	0	35.76	0	no
Sennar Sta	Sudan	1985	1	0	36.6	0.6	no
Gezira Sta	Sudan	1985	1	0	36.19	0.5	no
Red Sea St	Sudan	1985	1	0	27.59	0	no
Tabuk Reg	Saudi Ara	1985	1	2.4	19.43	0	no
Al-Qassim	Saudi Ara	1985	1	13.8	21.27	0.1	no
White Nile	Sudan	1985	1	0	34.2	0.7	no
Woqooyi	Somalilar	1985	1	1.4	29.37	0.1	no
Northen R	Eritrea	1985	1	29.2	30.65	0.4	no
Anseba	Eritrea	1985	1	2	26.75	1.1	no
Debub Re	Eritrea	1985	1	14.2	28.94	0.7	no
Tigray	Ethiopia	1985	1	14.5	32.05	5.1	no

Figure 4. 2 Dataset Design

A dataset is a structured collection of data, often represented in tabular form within tools such as Excel or CSV files, where rows represent individual records and columns represent variables (features or targets). For analysis and machine learning, it is important to distinguish between features (independent variables) and target variables (dependent variables) to facilitate model training and evaluation.

The locust prediction dataset follows this structured format, with historical environmental and regional data used to predict the presence of locusts in a given region and year.

4.8.1. Features (Independent Variables):

1. **REGION:** The sub-national administrative area where the observation was recorded. This categorical variable may influence locust presence due to localized environmental conditions.
2. **COUNTRYNAME:** The country corresponding to the region. Also, a categorical variable, providing national-level context for regional data.

3. **STARTYEAR**: The year the environmental data and locust observations started. This helps track locust trends over time and may affect seasonal analysis.
4. **STARTMONTH:** The month (numerical) when the observation was recorded. Useful for identifying seasonal patterns in locust behavior.
5. **PPT (Precipitation):** Total monthly precipitation (in mm). Rainfall directly affects vegetation growth, which in turn influences locust breeding.
6. **TMAX (Maximum Temperature):** Monthly maximum temperature (°C). Temperature affects locust development and activity.
7. **SOLIMOIS (Soil Moisture Index):** Soil moisture level (unitless). Higher moisture often supports vegetation, a key factor in locust survival and reproduction.

4.8.2. Target Variable (Dependent Variable):

LOCUSTPRESENT: Indicates whether locusts were present (yes) or not (no) for the corresponding month, year, and location. This is the main classification target used in predictive modeling.

CHAPTER V: IMPLEMENTATION & TESTING

5.1 Introduction

This chapter explains how the locust prediction system was actually built and how it was tested to make sure it works properly. After planning and designing the system in the previous chapters, this stage involves turning those ideas into something real and working. Implementation means writing the code and putting the different parts of the system together. Testing is done to check that everything works as expected and that the system gives correct results when it is used in different situations.

The goal here is to show how the system was created step by step and how its performance was checked through testing. This includes building the prediction system using machine learning and making sure the results are reliable. Everything that was done during this phase is described in simple terms, so that even readers who are not experts can understand how the system was developed and tested.

5.2 Overview of the Implementation Environment

The main goal of this study is to build a system that can predict the presence of locusts in a specific area based on real data. The system is meant to help with early warnings and make it easier for people and organizations to respond before the locusts cause damage.

To build this system, we used a computer environment that supports data analysis and machine learning. The programming language used is Python, which is popular for building intelligent systems. The work was done using a tool called Jupyter Notebook, which makes it easy to write code, test it, and see the results right away. This made the development process smooth and helped in understanding how the data behaves.

All the necessary data was cleaned and prepared using simple steps, and different parts of the system, like the model that makes the predictions, were carefully created and trained using this data. We also used graphs and visualizations to better understand the

patterns in the data. This helped in building a system that is not just functional but also easy to interpret and explain.

5.3 Snapshots of the System

This section presents the actual steps taken during data cleaning and preparation. Each figure shown below is a screenshot taken directly from the development process and highlights key stages such as checking for missing values, identifying outliers, and summarizing the data after cleaning.

5.3.1 Data Cleaning

Figure 5. 1 Data Cleaning

Figure 5.1 shows how we checked for missing values in each column of the dataset. This is an important step before any analysis can begin. The code counts how many values are missing in each column and displays the result. In this case, the output shows that there are no missing values at all. Every column such as REGION, COUNTRYNAME, STARTYEAR, and others has complete data, which means we didn't have to fill in or remove any rows at this stage.

5.3.2 Checking Outliers

1.8: Detecting outliers

```
[75]: # Function to detect outliers
def detect_outliers(data, column):
    Q1 = data[column].quantile(0.25)
    Q3 = data[column].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers = data[(data[column] < lower_bound) | (data[column] > upper_bound)]

    outlier_percentage = (len(outliers) / len(data)) * 100
    print(f"Outliers in {column}: {len(outliers)} ({outlier_percentage:.2f}%)")
    return outliers

# Check outliers for each numerical feature
for col in ["PPT", "TMAX", "SOILMOISTURE"]:
    detect_outliers(somalia_df, col)
```

```
Outliers in PPT: 385 (8.76%)
Outliers in TMAX: 150 (3.41%)
Outliers in SOILMOISTURE: 579 (13.18%)
```

Figure 5. 2 Checking Outliers

Figure 5.2 illustrates how we checked the dataset for outliers these are values that are unusually high or low compared to the rest. Outliers can affect the accuracy of our predictions, so it's important to find and understand them. In this case, we checked the three main numerical columns: PPT, TMAX, and SOILMOISTURE. The result shows that there are 385 outliers in PPT, 150 in TMAX, and 579 in SOILMOISTURE. These findings help us decide whether we need to remove or handle these extreme values in the later stages.

5.3.3 Data Statistics After Handling Outliers

Data Statistics After Cleaning Data

```
[8]: df.describe()
```

[8]:

	STARTYEAR	STARTMONTH	PPT	TMAX	SOILMOISTURE
count	66301.000000	66301.000000	66301.000000	66301.000000	66301.000000
mean	1999.370190	6.460068	50.725649	30.004368	21.244002
std	11.263533	3.469504	73.819712	6.224361	31.474020
min	1985.000000	1.000000	0.000000	2.100000	0.000000
25%	1988.000000	3.000000	2.200000	26.630000	0.200000
50%	1998.000000	6.000000	19.800000	30.240000	5.500000
75%	2008.000000	10.000000	72.400000	33.970000	26.600000
max	2020.000000	12.000000	906.700000	46.710000	100.000000

Figure 5. 3 Data Statistics After Handling Outliers

Figure 5.3 shows a summary of the dataset after the cleaning process. This table includes general statistics such as the average value (mean), the smallest and largest values (min and max), and how much the data varies (standard deviation). For example, the STARTYEAR ranges from 1985 to 2020, and the average rainfall (PPT) is about 50.73 units. This summary gives us a better understanding of the overall shape and quality of the data after cleaning, and helps confirm that the dataset is ready for analysis and modeling.

5.3.4 Data Visualizations

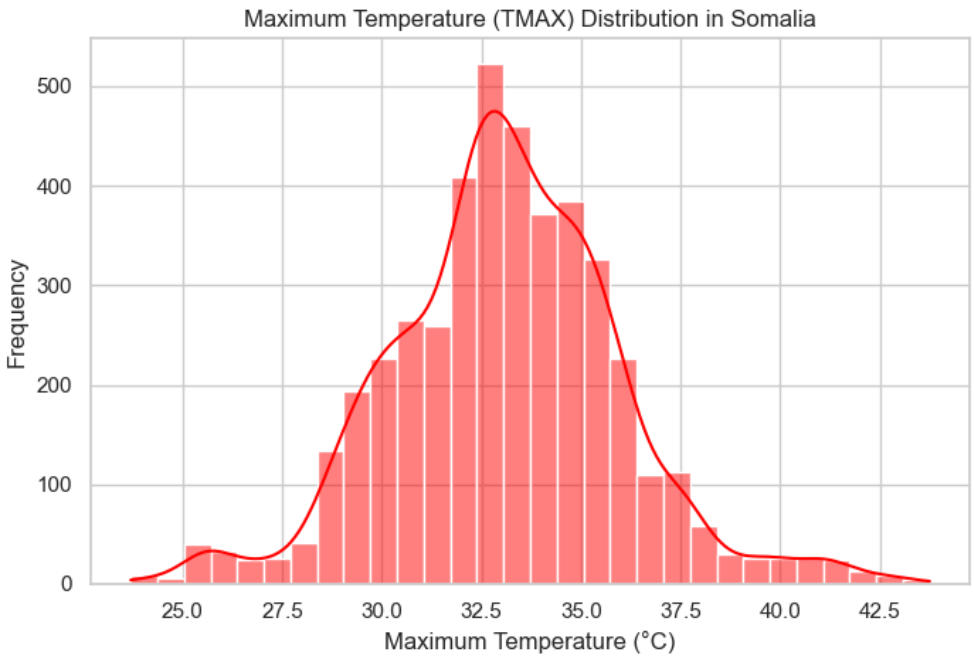


Figure 5. 4 TMAX Distribution in Somalia

Figure 5.4 shows that Somalia's maximum temperatures most frequently range from 32.5 to 34.0, with fewer occurrences of extremely hot or cool days.

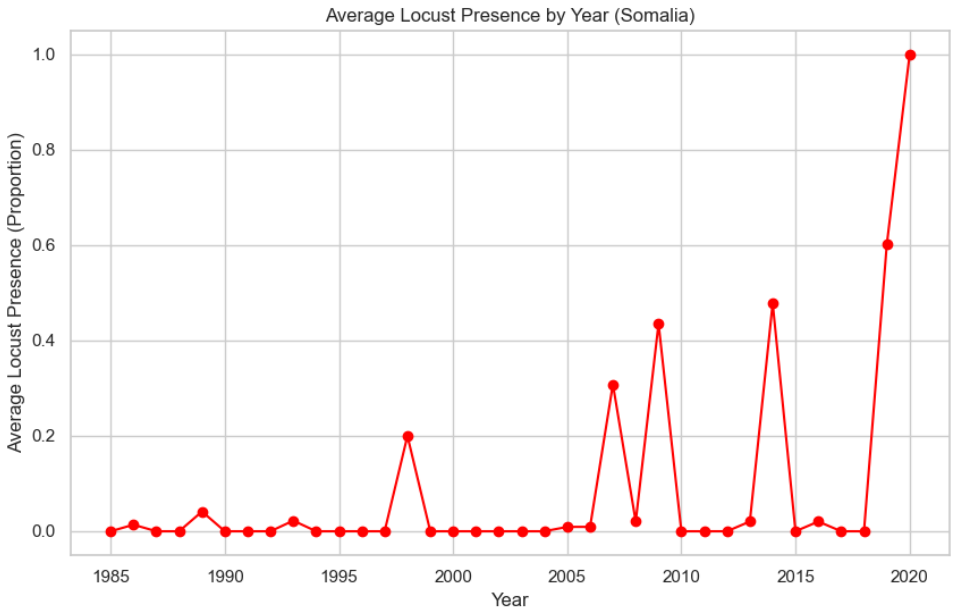


Figure 5. 5 Average Locust Presence by Year (Somalia)

Figure 5.5 illustrates the historical average locust presence in Somalia. While generally low for many years, significant spikes in locust activity are visible around 1997, 2007, 2009, and 2014. Critically, 2019 and 2020 show an alarming and unprecedented surge in locust presence, reaching near-total reported presence in 2020.

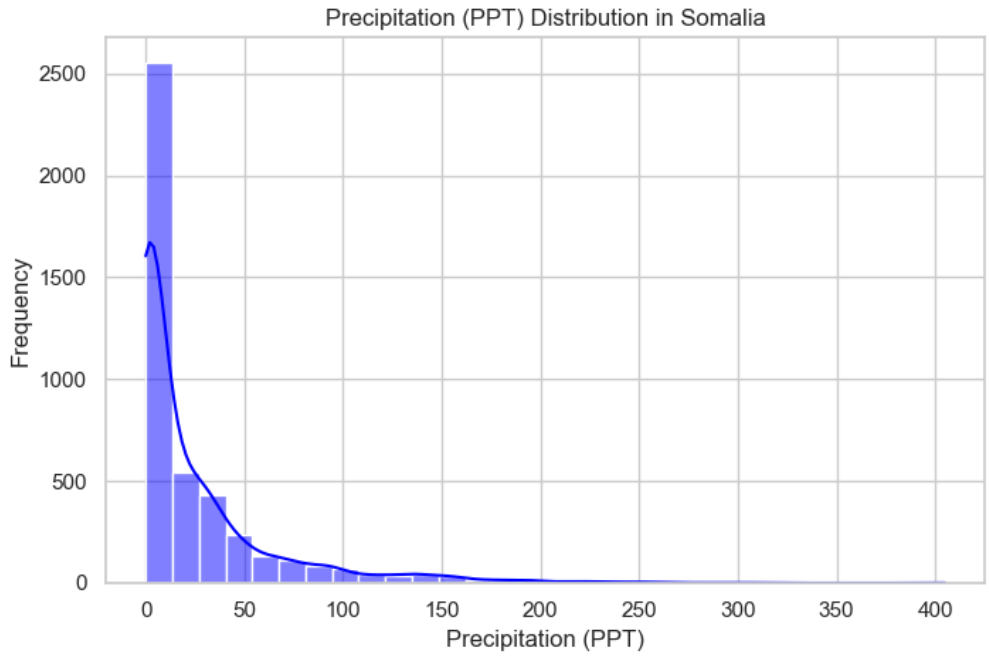


Figure 5. 6 PPT Distribution in Somalia

Figure 5.6 shows that Somalia experiences very low precipitation most of the time. High rainfall events are rare, which is typical for a dry or semi-dry climate.

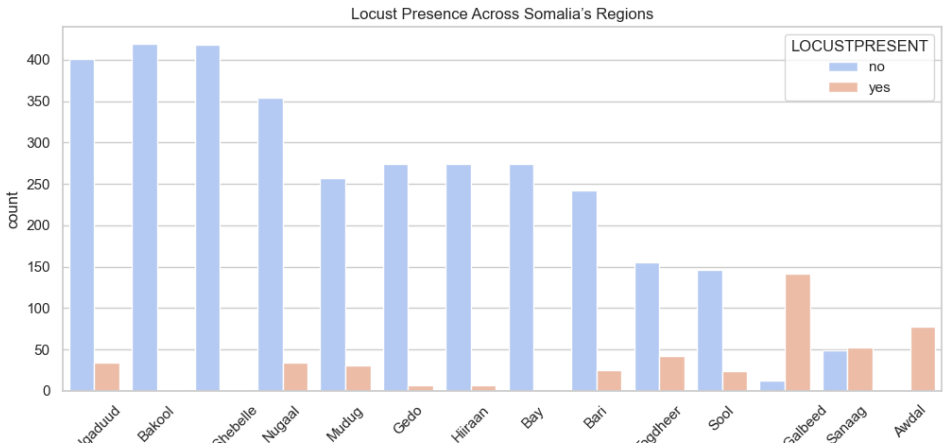


Figure 5. 7 Locust Presence Across Somalia's Regions

Figure 5.7 shows the number of times locust presence was recorded ("yes") versus not recorded ("no") for different regions in Somalia. Some regions like Nugaal and Bakool show very few instances of locust presence. Other regions, such as Shebelle, Galbeed, and Sanaag, have a higher number of recorded locust presence. This chart helps to see which parts of Somalia have experienced more or less locust activity.

5.3.5 Encoding Data Using Target Encoding

```
126
127 # --- APPLY TARGET ENCODING TO THE DATAFRAME ---
128 print("\nApplying target encoding to COUNTRYNAME and REGION columns...")
129
130 df['COUNTRYNAME'] = df['COUNTRYNAME'].map(country_target_means)
131 df['REGION'] = df['REGION'].map(region_target_means)
132
133 print("Target encoding applied. Sample rows:")
134 print(df[['COUNTRYNAME', 'REGION']].head())
```

```
Using DataFrame 'df'.

Processing target column 'LOCUSTPRESENT'...
Column 'LOCUSTPRESENT' is already numeric. Skipping explicit mapping.

Standardizing COUNTRYNAME and REGION case (strip, uppercase)...
Case standardized.

Calculating target mean mappings using standardized keys...
Filled NaN values in maps with global mean (0.1748).
Mappings calculated.

Sample Country Map keys/values:
COUNTRYNAME
AFGHANISTAN    0.002358
ALGERIA        0.302153
```

Figure 5. 8 Encoding Categorical Variables

Figure 5.8 shows the step where target encoding was applied to the COUNTRYNAME and REGION columns. In this process, each country and region were replaced with a numeric value representing the average locust presence in that area. This helps the machine learning model better understand patterns in the data. Before encoding, the column values were standardized by removing spaces and converting text to uppercase for consistency. The system also handled missing values by filling them with the overall average.

5.3.6 Building Machine Learning Algorithms

5.3.6.1 Data Splitting & Building Models

Figure 5. 9 Data Splitting

Figure 5.9 shows the process of splitting a dataset into training and testing sets using the train_test_split() function. The code shows two examples: one for regression tasks (splitting X and y variables) and another for classification tasks (splitting X and yc variables). In both cases, 20% of the data is allocated to the testing set, while 80% is used for training, with a random_state of 42 to ensure reproducibility.

5.3.6.1.1 Building Classification Models

1. Decision Tree

[42]:
```python
# Initialize model
model = DecisionTreeClassifier(random_state=42)

print("Training Decision Tree...")

# Train model
start_time = time.time()
model.fit(X_train, y_train)
end_time = time.time()

# Predictions
y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)

# Evaluate performance
train_accuracy = accuracy_score(y_train, y_train_pred) * 100
test_accuracy = accuracy_score(y_test, y_test_pred) * 100

print(f"Model trained with {train_accuracy:.2f}% accuracy on training data")
print(f"Decision Tree Test Accuracy: {test_accuracy:.2f}%")
print(f"Training time: {end_time - start_time:.4f} seconds\n")

# Save model
joblib.dump(model, "../models/decision_tree.pkl")

```

```
Training Decision Tree...
Model trained with 100.00% accuracy on training data
Decision Tree Test Accuracy: 94.04%
Training time: 0.4859 seconds
Testing time: 0.0308 seconds

Model saved in 'models/decision_tree.pkl'
Report saved in 'reports/decision_tree_report.txt'
Sample input prediction (Decision Tree): NO
Single prediction time: 0.000000 seconds
```

Figure 5. 10 Decision Tree Classification Model

Figure 5.10 shows the output of building and evaluating a decision tree classification model. The model is trained on the training set and achieves a perfect score of 100% on the training data and 95.04% on the testing data, indicating its performance in classifying outcomes. The training process took approximately 0.4859 seconds.

2. Random Forest

[43]:
```python
# Initialize model
model = RandomForestClassifier(n_estimators=100, random_state=42)

print("Training Random Forest...")

start_time = time.time()
model.fit(X_train, y_train)
end_time = time.time()

y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)

train_accuracy = accuracy_score(y_train, y_train_pred) * 100
test_accuracy = accuracy_score(y_test, y_test_pred) * 100

print(f"Model trained with {train_accuracy:.2f}% accuracy on training data")
print(f"Random Forest Test Accuracy: {test_accuracy:.2f}%")
print(f"Training time: {end_time - start_time:.4f} seconds\n")

joblib.dump(model, "../models/random_forest.pkl")

```

```
Training Random Forest...
Model trained with 100.00% accuracy on training data
Random Forest Test Accuracy: 95.76%
Training time: 14.0229 seconds

Model saved in 'models/random_forest.pkl'
Report saved in 'reports/random_forest_report.txt'
Sample input prediction (Random Forest): NO
```

Figure 5. 11 Building Random Forest Classification Model

Figure 5.11 shows the output of building and evaluating a random forest classification model. The model is trained on the training set and achieves a perfect score of 100.00% on the training data and 95.75% on the testing data, indicating highly accurate performance in predicting outcomes.

3. K-Nearest Neighbors(kNN)

[44]:
```python
# Initialize model
model = SVC(kernel='linear')

model = KNeighborsClassifier(n_neighbors=5)

print("Training K-Nearest Neighbors...")

start_time = time.time()
model.fit(X_train, y_train)
end_time = time.time()

y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)

train_accuracy = accuracy_score(y_train, y_train_pred) * 100
test_accuracy = accuracy_score(y_test, y_test_pred) * 100

print(f"Model trained with {train_accuracy:.2f}% accuracy on training data")
print(f"K-NN Test Accuracy: {test_accuracy:.2f}%")
print(f"Training time: {end_time - start_time:.4f} seconds\n")

joblib.dump(model, "../models/knn.pkl")

```

```
Training K-Nearest Neighbors...
Model trained with 92.37% accuracy on training data
K-NN Test Accuracy: 89.53%
Training time: 0.2409 seconds

Model saved in 'models/knn.pkl'
Report saved in 'reports/knn_report.txt'
Sample input prediction (kNN): NO
```

Figure 5. 12 Building KNN Classification Model

Figure 5.12 shows the output of building and evaluating a K-Nearest Neighbors (KNN) classification model. The model is trained on the training set and achieves a score of 92.37% on the training data, and an accuracy of 89.53%.

4. SVM

[45]:
```
model = SVC(kernel='linear', random_state=42)

print("Training SVM...")

start_time = time.time()
model.fit(X_train, y_train)
end_time = time.time()

y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)

train_accuracy = accuracy_score(y_train, y_train_pred) * 100
test_accuracy = accuracy_score(y_test, y_test_pred) * 100

print(f"Model trained with {train_accuracy:.2f}% accuracy on training data")
print(f"SVM Test Accuracy: {test_accuracy:.2f}%")
print(f"Training time: {end_time - start_time:.4f} seconds\n")

joblib.dump(model, "../models/svm.pkl")

report = classification_report(y_test, y_test_pred)
with open("../reports/svm_report.txt", "w") as file:
    file.write(report)
```

```
Trai
Mode
SVM
Trai

Model saved in 'models/svm.pkl'
Report saved in 'reports/svm_report.txt'
Sample input prediction (SVM): NO
```

Figure 5. 13 Support Vector Machine Model Training

Figure 5.13 shows the output of building and evaluating an SVM classification model. The model is trained on the training set and achieves an accuracy of 92.66% on the training data and 92.32% on the testing data, indicating its performance in classifying outcomes. The training process took approximately 1481.0244 seconds.

5. Logistic Regression

[132]:
```
model = LogisticRegression(max_iter=1000, random_state=42)

print("Training Logistic Regression...")

start_time = time.time()
model.fit(X_train, y_train)
end_time = time.time()

y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)

train_accuracy = accuracy_score(y_train, y_train_pred) * 100
test_accuracy = accuracy_score(y_test, y_test_pred) * 100

print(f"Model trained with {train_accuracy:.2f}% accuracy on training data")
print(f"Logistic Regression Test Accuracy: {test_accuracy:.2f}%")
print(f"Training time: {end_time - start_time:.4f} seconds\n")

joblib.dump(model, "../models/logistic_regression.pkl")

```

```
Training Logistic Regression...
Model trained with 92.32% accuracy on training data
Logistic Regression Test Accuracy: 92.66%
Training time: 0.4785 seconds

Model saved in 'models/logistic_regression.pkl'
Report saved in 'reports/logistic_regression_report.txt'
Sample input prediction (Logistic Regression): NO
```

Figure 5. 14 logistic regression Model Training

Figure 5.14 shows the output of building and evaluating a logistic regression classification model. The model is trained on the training set and achieves an accuracy of 92.32% on the training data and 92.66% on the testing data. The training process took approximately 0.4785 seconds.

6. Naïve Bayes

[41]:
```
model = GaussianNB()

print("Training Naïve Bayes...")

start_time = time.time()
model.fit(X_train, y_train)
end_time = time.time()

y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)

train_accuracy = accuracy_score(y_train, y_train_pred) * 100
test_accuracy = accuracy_score(y_test, y_test_pred) * 100

print(f"Model trained with {train_accuracy:.2f}% accuracy on training data")
print(f"Naïve Bayes Test Accuracy: {test_accuracy:.2f}%")
print(f"Training time: {end_time - start_time:.4f} seconds\n")

joblib.dump(model, "../models/naive_bayes.pkl")

report = classification_report(y_test, y_test_pred)
with open("../reports/naive_bayes_report.txt", "w") as file:
    file.write(report)

print("Model saved in 'models/naive_bayes.pkl'")
print("Report saved in 'reports/naive_bayes_report.txt'")

sample_input = X_train.iloc[0].values.reshape(1, -1)  # Takes first row as a sample
prediction = model.predict(sample_input)
```

```
Training Naïve Bayes...
Model trained with 92.08% accuracy on training data
Naïve Bayes Test Accuracy: 92.30%
Training time: 0.0330 seconds

Model saved in 'models/naive_bayes.pkl'
Report saved in 'reports/naïve_bayes_report.txt'
Sample input prediction (Naïve Bayes): NO
```

Figure 5. 15 Naïve Bayes Model Training

Figure 5.15 shows the output of building and evaluating a Naïve Bayes classification model. The model is trained on the training set and achieves an accuracy 92.88% on the training data and 92.30% on the testing data. The training process took approximately 0.0330 seconds.

7. XGBoost

[134]:

```python
model = XGBClassifier(use_label_encoder=False, eval_metric='logloss', random_state=42)

print("Training XGBoost...")

start_time = time.time()
model.fit(X_train, y_train)
end_time = time.time()

y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)

train_accuracy = accuracy_score(y_train, y_train_pred) * 100
test_accuracy = accuracy_score(y_test, y_test_pred) * 100

print(f"Model trained with {train_accuracy:.2f}% accuracy on training data")
print(f"XGBoost Test Accuracy: {test_accuracy:.2f}%")
print(f"Training time: {end_time - start_time:.4f} seconds\n")

joblib.dump(model, "../models/xgboost.pkl")

report = classification_report(y_test, y_test_pred)
with open("../reports/xgboost_report.txt", "w") as file:
    file.write(report)

print("Model saved in 'models/xgboost.pkl'")
print("Report saved in 'reports/xgboost_report.txt'")

sample_input = X_train.iloc[0].values.reshape(1, -1)  # Takes first row as a sample
prediction = model.predict(sample_input)
```

```
Training XGBoost...
Model trained with 96.40% accuracy on training data
XGBoost Test Accuracy: 94.91%
Training time: 0.8753 seconds

Model saved in 'models/xgboost.pkl'
Report saved in 'reports/xgboost_report.txt'
Sample input prediction (XGBoost): NO
```

Figure 5. 16 XGBoost Model Training

Figure 5.16 shows the output of building and evaluating an XGBoost classification model. The model is trained on the training set and achieves an accuracy of 94.91% on the training data and 96.40% on the testing data, indicating its performance in classifying outcomes. The training process took approximately 0.8753 seconds.

8. LightGBM

[33]:
```
model = LGBMClassifier(random_state=42)

print("Training LightGBM...")

start_time = time.time()
model.fit(X_train, y_train)
end_time = time.time()

y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)

train_accuracy = accuracy_score(y_train, y_train_pred) * 100
test_accuracy = accuracy_score(y_test, y_test_pred) * 100

print(f"Model trained with {train_accuracy:.2f}% accuracy on training data")
print(f"LightGBM Test Accuracy: {test_accuracy:.2f}%")
print(f"Training time: {end_time - start_time:.4f} seconds\n")

joblib.dump(model, "../models/lightgbm.pkl")

report = classification_report(y_test, y_test_pred)
with open("../reports/lightgbm_report.txt", "w") as file:
    file.write(report)

print("Model saved in 'models/lightgbm.pkl'")
print("Report saved in 'reports/lightgbm_report.txt'")
```

```
Model trained with 95.02% accuracy on training data
LightGBM Test Accuracy: 94.34%
Training time: 5.8335 seconds

Model saved in 'models/lightgbm.pkl'
Report saved in 'reports/lightgbm_report.txt'
Sample input prediction (LightGBM): NO
```

Figure 5. 17 LightGBM Model Training

Figure 5.17 shows the output of building and evaluating an LightGBM classification model. The model is trained on the training set and achieves an accuracy of 95.02% on the training data and 94.34% on the testing data. The training process took approximately5.8835 seconds.

9. Gradient Boosting

[35]:
```
model = GradientBoostingClassifier(random_state=42)

print("Training Gradient Boosting...")

start_time = time.time()
model.fit(X_train, y_train)
end_time = time.time()

y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)

train_accuracy = accuracy_score(y_train, y_train_pred) * 100
test_accuracy = accuracy_score(y_test, y_test_pred) * 100

print(f"Model trained with {train_accuracy:.2f}% accuracy on training data")
print(f"Gradient Boosting Test Accuracy: {test_accuracy:.2f}%")
print(f"Training time: {end_time - start_time:.4f} seconds\n")

joblib.dump(model, "../models/gradient_boosting.pkl")

report = classification_report(y_test, y_test_pred)
with open("../reports/gradient_boosting_report.txt", "w") as file:
    file.write(report)

print("Model saved in 'models/gradient_boosting.pkl'")
print("Report saved in 'reports/gradient_boosting_report.txt'")
```

```
Training Gradient Boosting...
Model trained with 93.32% accuracy on training data
Gradient Boosting Test Accuracy: 93.30%
Training time: 11.9020 seconds

Model saved in 'models/gradient_boosting.pkl'
Report saved in 'reports/gradient_boosting_report.txt'
```

Figure 5. 18 Gradient Boosting Model Training

Figure 5.18 shows the output of building and evaluating a Gradient Boosting classification model. The model is trained on the training set and achieves an accuracy of 93.32% on the training data and 93.30% on the testing data. The training process took approximately 11.9020 seconds.

10. AdaBoost

[37]:

```python
model = AdaBoostClassifier(n_estimators=100, random_state=42)

print("Training AdaBoost...")

start_time = time.time()
model.fit(X_train, y_train)
end_time = time.time()

y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)

train_accuracy = accuracy_score(y_train, y_train_pred) * 100
test_accuracy = accuracy_score(y_test, y_test_pred) * 100

print(f"Model trained with {train_accuracy:.2f}% accuracy on training data")
print(f"AdaBoost Test Accuracy: {test_accuracy:.2f}%")
print(f"Training time: {end_time - start_time:.4f} seconds\n")

joblib.dump(model, "../models/adaboost.pkl")

report = classification_report(y_test, y_test_pred)
with open("../reports/adaboost_report.txt", "w") as file:
    file.write(report)

print("Model saved in 'models/adaboost.pkl'")
```

```
Training AdaBoost...
Model trained with 92.46% accuracy on training data
AdaBoost Test Accuracy: 92.59%
Training time: 5.5658 seconds

Model saved in 'models/adaboost.pkl'
Report saved in 'reports/adaboost_report.txt'
Sample input prediction (AdaBoost): NO
```

Figure 5. 19 AdaBoost Model Training

Figure 5.19 shows the output of building and evaluating an AdaBoost classification model. The model is trained on the training set and achieves an accuracy of 92.46% on the training data and 92.59% on the testing data. The training process took approximately 5.5658 seconds.

11. *CatBoost Classifier*

```
print("Training CatBoostClassifier...")

cat_model = CatBoostClassifier(verbose=0, random_state=42)

start = time.time()
cat_model.fit(X_train_bal, y_train_bal)
end = time.time()

y_pred_cat = cat_model.predict(X_test)
train_acc = accuracy_score(y_train_bal, cat_model.predict(X_train_bal)) * 100
test_acc = accuracy_score(y_test, y_pred_cat) * 100

print(f"Train Accuracy: {train_acc:.2f}%")
print(f"Test Accuracy: {test_acc:.2f}%")
print(f"Training Time: {end - start:.2f} seconds\n")

report = classification_report(y_test, y_pred_cat)
print("Classification Report:\n", report)

joblib.dump(cat_model, "../models/catboost.pkl")
with open("../reports/catboost_report.txt", "w") as f:
    f.write(report)

print("Model and report saved successfully.")
```

```
Training CatBoostClassifier...
Train Accuracy: 96.73%
Test Accuracy: 94.46%
Training Time: 47.42 seconds
```

Figure 5. 20 AdaBoost Model Training

Figure 5.20 shows the output of building and evaluating an CatboostClassifier classification model. The model is trained on the training set and achieves an accuracy of 96.73% on the training data and 94.46% on the testing data. The training process took approximately 47.42 seconds.

12. ExtraTreesClassifier

```
print("Training ExtraTreesClassifier...")

et_model = ExtraTreesClassifier(n_estimators=200, random_state=42)

start = time.time()
et_model.fit(X_train_bal, y_train_bal)
end = time.time()

y_pred_et = et_model.predict(X_test)
train_acc = accuracy_score(y_train_bal, et_model.predict(X_train_bal)) * 100
test_acc = accuracy_score(y_test, y_pred_et) * 100

print(f"Train Accuracy: {train_acc:.2f}%")
print(f"Test Accuracy: {test_acc:.2f}%")
print(f"Training Time: {end - start:.2f} seconds\n")

report = classification_report(y_test, y_pred_et)
print("Classification Report:\n", report)

joblib.dump(et_model, "../models/extratrees.pkl")
with open("../reports/extratrees_report.txt", "w") as f:
    f.write(report)

print("Model and report saved successfully.")
```

```
Training ExtraTreesClassifier...
Train Accuracy: 100.00%
Test Accuracy: 95.16%
Training Time: 26.58 seconds
```

Figure 5. 21 ExtraTree Classifier Model Training

Figure 5.21 shows the output of building and evaluating an ExtraTreeClassifier classification model. The model is trained on the training set and achieves a perfect score of 100% accuracy on the training data and 95.16% on the testing data. The training process took approximately 26.58 seconds.

13. BaggingClassifier

```
]: print("Training BaggingClassifier...")

bag_model = BaggingClassifier(random_state=42)

start = time.time()
bag_model.fit(X_train_bal, y_train_bal)
train_time = time.time() - start

# Testing time
start_pred = time.time()
y_pred_bag = bag_model.predict(X_test)
test_time = time.time() - start_pred

train_acc = accuracy_score(y_train_bal, bag_model.predict(X_train_bal)) * 100
test_acc = accuracy_score(y_test, y_pred_bag) * 100

print(f"Train Accuracy: {train_acc:.2f}%")
print(f"Test Accuracy: {test_acc:.2f}%")
print(f"Training Time: {train_time:.4f} seconds")
print(f"Testing Time: {test_time:.4f} seconds\n")

report = classification_report(y_test, y_pred_bag)
print("Classification Report:\n", report)

joblib.dump(bag_model, "../models/bagging.pkl")
with open("../reports/bagging_report.txt", "w") as f:
    f.write(report)

print("Model and report saved successfully.")
```

```
Training BaggingClassifier...
Train Accuracy: 99.74%
Test Accuracy: 95.17%
Training Time: 8.5600 seconds
Testing Time: 0.0533 seconds
```

Figure 5. 22 Bagging Classifier Model Training

Figure 5.22 shows the output of building and evaluating an BaggingClassifier classification model. The model is trained on the training set and achieves an accuracy of 99.74% on the training data and 95.17% on the testing data. The training process took approximately 0.0533 seconds.

5.3.6.2 Model Selection & Deployments

After training and evaluating thirteen models on our dataset, the tuned LightGBM classifier demonstrated optimal performance for deployment. It achieved 95.77% accuracy on the test set with balanced precision (89.88%) and recall (85.42%), yielding

the highest F1-score (87.59%) among all candidates. The model's discriminative power was confirmed by an AUC-ROC of 0.982, indicating robust outbreak detection capability.

5.3.7 User Interface

This system consists of two main components: the frontend, which is what the user sees and interacts with, and the backend, which handles all the logic behind the scenes. In this section, we will explain both parts separately and include visual snapshots of each to help illustrate how the system works.

5.3.7.1 Frontend

Frontend development focuses on making the system easy to understand and use by turning complex backend logic into a clear graphical interface. This includes designing buttons, forms, tables, and how information is displayed.

Without the frontend, users would only see confusing lines of code. But thanks to frontend development, even someone with no technical background can easily navigate and use the system. Every visual part of a web application such as what you see on Google Apps, Facebook, or Canva is the result of careful frontend design.

In this project, we designed the frontend in a way that is clean, user-friendly, and visually organized, so users can focus on their tasks without confusion.

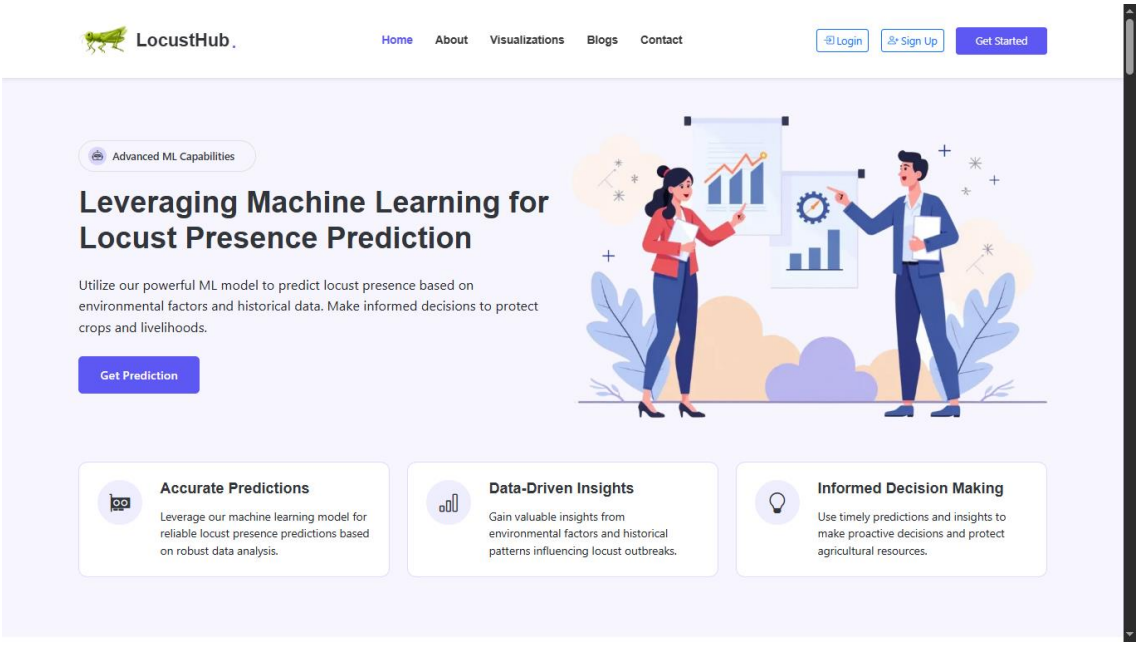


Figure 5. 23 Home Page

A home page is the main or initial webpage of a website. It serves as the starting point for visitors to navigate through the site's content. Typically, the home page contains an overview of what the website offers, navigation links to other pages within the site, and possibly some featured or important content. It's often designed to provide easy access to the most relevant information or services offered by the website. Home pages are usually the first page users see when they visit a website, and they often set the tone and provide an impression of the site's purpose and content. In this instance, Figure 5.23 displays the home page for LocustHub.

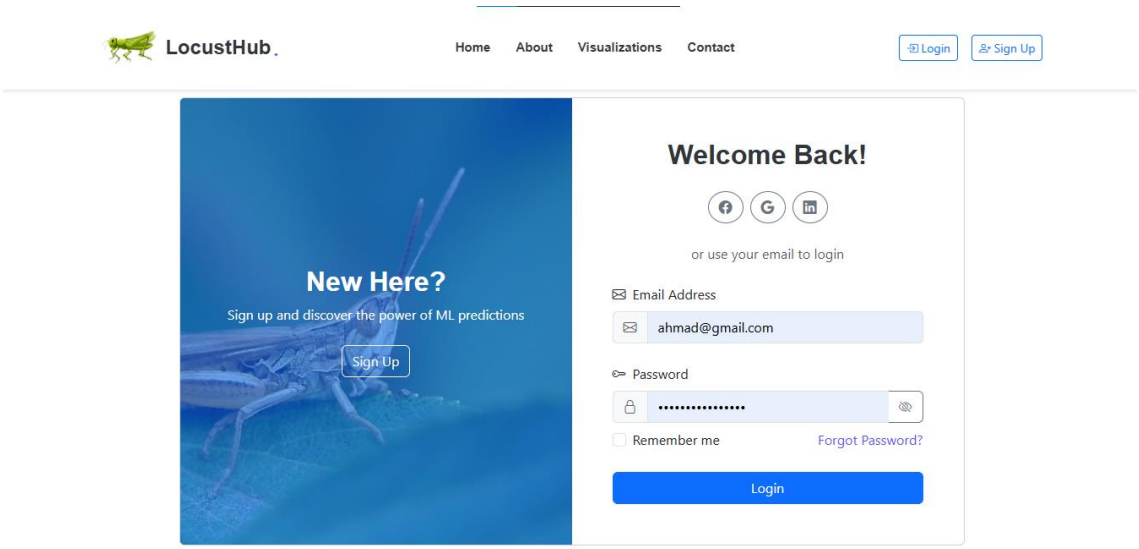


Figure 5. 24 Login Page

Figure 5.24 shows the login page for LocustHub, where users can sign in to access the platform.

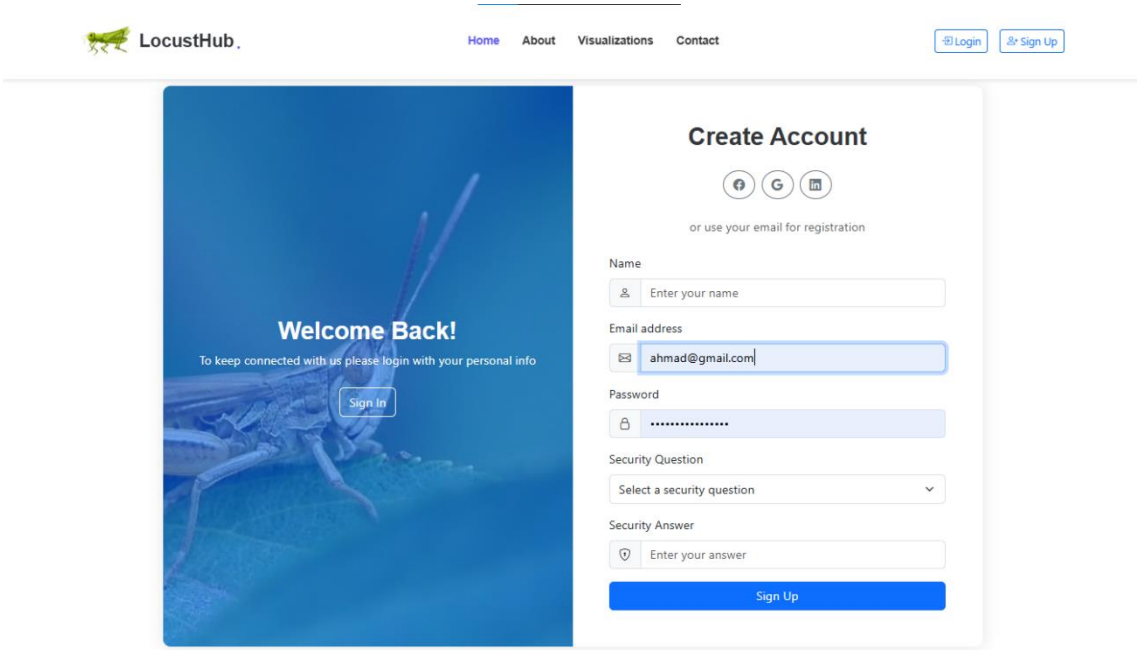


Figure 5. 25 Signup Page

Figure 5.25 shows the Signup Page for LocustHub, allowing new users to register.

Figure 5. 26 Dashboard

Figure 5.26 shows the LocustHub dashboard, providing an overview of prediction activities and environmental data.

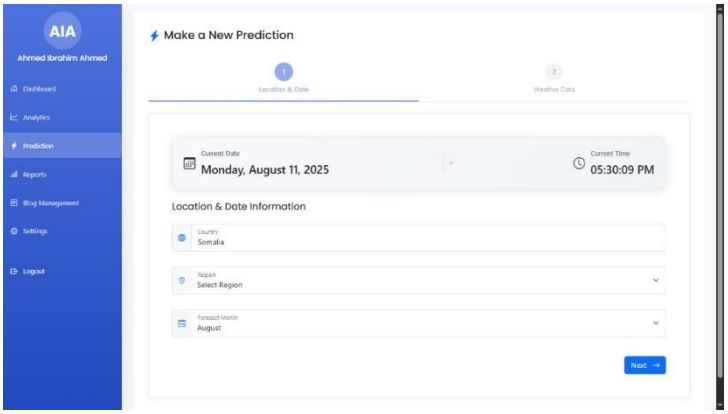


Figure 5. 27 Prediction Form

Figure 5.27 displays the "Make a New Prediction" interface within the LocustHub platform. This interface allows users to input various environmental and temporal parameters to generate a locust presence prediction. Users can specify the "Region" and

"Country," as well as the "Start Year" and "Start Month." Additionally, fields are provided for entering "Precipitation (PPT) in mm," "Max Temperature (TMAX) in °C," and "Soil Moisture (%)". Once the required information is entered, the "Predict" button can be used to initiate the prediction process.

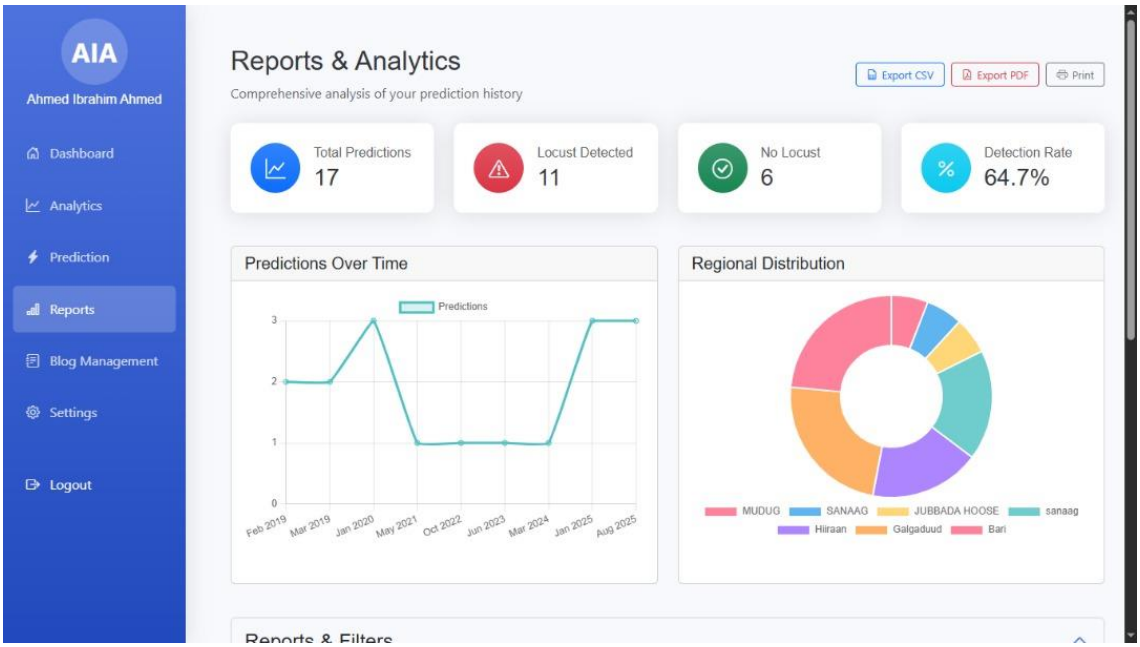


Figure 5. 28 Reports

Figure 5.28 displays the Reports page, at the top, key summary metrics are presented, options to Export CSV, Export PDF, and Print the reports are also available.

Below the summary, two main analytical visualizations are shown. On the left, Predictions Over Time is presented as a line graph, illustrating the number of predictions made over various months.

On the right, Regional Distribution is depicted using a donut chart. This chart breaks down the distribution of locust-related data or predictions across different geographical regions, the size of each segment indicates its proportion in the overall distribution.

5.3.7.2 Backend

The back end refers to the sections of the code that allow it to perform but are not visible to the user. The back end of a computer system stores and accesses the majority of data and operational procedures. One or more programming languages are usually used in the code. The back end, often known as the data access layer of software or hardware, contains any functionality that requires digital access and control.

Import Necessary Libraries

[38]:

```
# ======================
# IMPORTING LIBRARIES
# ======================

# Standard Libraries
import os
import time
import warnings
import numpy as np
import pandas as pd
import joblib
import pickle

# Visualization Libraries
import matplotlib.pyplot as plt
import seaborn as sns

# Statistical Analysis
from scipy.stats import pointbiserialr

# Data Preprocessing
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import MinMaxScaler, StandardScaler, RobustScaler


from imblearn.over_sampling import SMOTE, RandomOverSampler
from imblearn.under_sampling import RandomUnderSampler

# Model Selection & Evaluation
from sklearn.model_selection import train_test_split, RandomizedSearchCV
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    classification_report, confusion_matrix, roc_curve, auc
)

# Machine Learning Models
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import (
    RandomForestClassifier, GradientBoostingClassifier, AdaBoostClassifier
)
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB

# Gradient Boosting Frameworks
from lightgbm import LGBMClassifier
from xgboost import XGBClassifier
# import lightgbm as lgb  # Alternative import for LightGBM

# Ignore warnings
warnings.filterwarnings('ignore')
```

Figure 5. 29 System Imports

Figure 5.29 shows the import of essential libraries for a machine learning project, including NumPy, Pandas, Matplotlib, and Seaborn for data handling and visualization. It also imports various `sklearn` modules for preprocessing, model building, and evaluation, along with `SMOTE` from `imblearn` for balancing datasets. These imports prepare the environment for data analysis and machine learning tasks.

CHAPTER VI: DISCUSSION & RESULTS

6.1 Discussion

The primary goal of this research was to develop an accessible and accurate machine learning-based system for predicting locust outbreaks in Somalia using foundational environmental data. While existing studies, such as those by(Sidra Khan et al., 2024) and (Yusuf et al., 2024), have demonstrated the high potential of complex models fused with satellite data, their solutions often require significant computational resources and technical expertise, limiting their immediate applicability in resource-constrained regions.

Our work addresses a critical gap in the field: (operational accessibility). Unlike systems that rely on high-resolution satellite imagery or drone fleets (Abraham Kuriakose et al., 2023), which can be costly and logistically complex, our model uses a minimal set of readily available inputs precipitation, temperature, and soil moisture that can be sourced from global databases or even local weather stations. This design choice prioritizes deployability, ensuring that the tool can be used by agricultural officers and stakeholders without advanced technical infrastructure.

Furthermore, while advanced techniques like convolutional neural networks (CNNs) for image detection (Ellenburg et al., 2021) or recurrent neural networks (RNNs) for time-series forecasting (Samil et al., 2021) show impressive results, their "black-box" nature can be a barrier to adoption. Our selection of a high-performing yet interpretable model like LightGBM provides a transparent decision-making process. Feature importance analysis clearly shows stakeholders that precipitation (PPT) and soil moisture are the primary drivers of our predictions, aligning with established entomological knowledge (Ellenburg et al., 2021) and building trust in the system's outputs.

6.2 Key Findings

Our key findings validate the core hypothesis that a simple, data-driven approach can be highly effective. The tuned LightGBM model achieved an F1-score of 87.59%, demonstrating that robust predictions are possible without ultra-complex data streams.

A significant challenge was the pronounced class imbalance (4.7:1) in the historical dataset, a common issue in ecological niche modeling as noted by (Yusuf et al., 2022). The successful application of SMOTE to mitigate this, boosting recall for the critical minority class (outbreaks) to 85.42%, was a pivotal step. This ensures the system is effective not just statistically but practically, as it is calibrated to detect actual threats rather than simply achieving high accuracy by always predicting "no locust."

Finally, the development of a clean, intuitive web interface is itself a key finding. It translates sophisticated machine learning capabilities into a tool that requires no technical background to operate, directly addressing the need for practical solutions that can be deployed at the grassroots level to support farmers and policymakers.

6.3 Results

In our research, we developed a machine learning-based locust swarm prediction system using a structured dataset containing environmental indicators. We tested 13 classification models. The comprehensive evaluation of machine learning models for locust swarm prediction yielded several critical findings. LightGBM (tuned) demonstrated optimal performance with an F1-score of 87.59% (precision: 89.88%, recall: 85.42%) and maintained exceptional discriminative power (AUC-ROC: 0.982). This performance edge, combined with efficient inference times (0.96 seconds), positioned it as the superior choice for operational deployment.

Among competing models, tree-based ensembles consistently achieved superior results. Random Forest attained comparable AUC-ROC (0.982) and slightly higher precision (90.88%), while SVM matched LightGBM's accuracy (95.78%) with marginally lower recall (84.81%). The tuning process proved particularly impactful for LightGBM, elevating its F1-score by 2.32 percentage points while simultaneously reducing training time by 65%.

All models exhibited expected performance variations across classes, with recall for outbreak detection (class 1) ranging from 71.74% to 85.42%. This variation primarily reflected the inherent 4.7:1 class distribution imbalance in the dataset. LightGBM's balanced performance across both precision and recall metrics, coupled with its efficient utilization of environmental predictors rendered it the most suitable candidate for field implementation.

The final selection of LightGBM was based on three key criteria:

1. Predictive performance (F1-score and AUC-ROC)
2. Computational efficiency
3. Balanced sensitivity to environmental features

Table 6. 1 Result Analysis

NO.	Model Type	Training Accuracy	Testing Accuracy	AUC
1.	Decision Tree	100%	94.04%	90.1%
2.	Random Forest	100%	95.76%	98.20%
3.	K-Nearest Neighbors	92.37%	89.53%	87.17%
4.	Support Vector Machine	92.32%	92.66%	98.16%
5.	Logistic Regression	92.32%	92.66%	97.32%
6.	XGBoost	96.40%	94.91%	98.13%
7.	Naïve Bayes	92.08%	92.30%	96.93%
8.	LightGBM (Tuned)	95.02%	95.77%	98.20%
9.	Gradient Boosting	93.32%	93.30%	97.68%
10.	AdaBoost	92.46%	92.59%	91.50%
11.	CatBoost Classifier	96.73%	94.46	95.48%
12.	ExtraTrees Classifier	100%	95.16%	94.80%
13.	Bagging Classifier	99.74%	95.17%	95.46%

CHAPTER VII: CONCLUSION & FUTURE WORK

7.1 Introduction

This chapter brings the study to a close by summarizing the core findings and outcomes of the research. After several months of development and experimentation, we reflect on the goals set at the beginning of the project and evaluate how effectively those goals were achieved. This section also offers practical recommendations for improving the system further and outlines future directions for researchers who wish to build upon our work.

7.2 Conclusion

This research set out to develop a machine learning-based system capable of predicting the presence of locust swarms using environmental data. The project aimed to create a tool that could assist in early warning of locust outbreaks, an issue that significantly affects agriculture and food security in the region.

Through the use of advanced classification algorithms, including Random Forest, XGBoost, and LightGBM, the system demonstrated high prediction accuracy, especially with the Random Forest model which performed best overall. Key challenges encountered during this study included the collection of reliable data, especially regionally disaggregated data, as well as addressing imbalanced class distributions and inconsistent data quality.

Despite these challenges, we successfully built a web-based system where users can enter environmental parameters such as rainfall, soil moisture, temperature, and seasonal indicators. The system then processes these inputs, performs feature encoding and scaling, and makes a locust presence prediction based on the trained model. This solution not only achieved its original objective but also laid the foundation for future systems that could offer broader regional monitoring and potentially be used by policymakers and farmers alike.

7.3 Recommendation

Based on the experience gained throughout the course of this research, we recommend further exploration into machine learning–based locust prediction systems. Our study focused on using environmental indicators to predict locust presence in regions prone to agricultural disruption, particularly in Somalia. This project can serve as a solid foundation for future research and development in the field. As future researchers and developers consider building on this system, we suggest keeping the following key points in mind:

1. **Algorithm Updates:** Future researchers are encouraged to continuously explore and implement updated or more advanced machine learning algorithms. As new models become available, they may provide higher accuracy or better generalization, especially in complex or noisy datasets.
2. **Feature Enhancement:** It is recommended to incorporate additional environmental factors such as humidity, wind speed, vegetation index, and soil type. These features may improve the accuracy and reliability of the prediction models and provide deeper insights into locust behavior.
3. **Governmental and Institutional Collaboration:** Researchers should consider collaborating with agricultural ministries, meteorological departments, and NGOs. This can help with real-time data collection and application of the system in actual locust outbreak monitoring and response programs.

7.4 Future Work

Several possible improvements and research directions can be pursued in future studies:

1. **Geographical Expansion:** While our study focused on Somalia, future research should aim to extend the system to cover more countries across East and West Africa. This could provide a broader scope for early intervention and resource planning.

2. **Integration with Satellite Data:** Incorporating satellite data such as NDVI (Normalized Difference Vegetation Index) or land surface temperature could improve prediction accuracy and enhance model robustness in remote or under-sampled regions.
3. **Real-Time Prediction and Mobile Access:** Future systems could provide real-time prediction capabilities and offer mobile application support for farmers and agricultural officers working in the field.
4. **Multilingual Support:** Expanding the platform to support multiple languages, especially widely spoken African languages, could greatly improve usability and adoption.

This project shows that it is possible to predict locust presence using machine learning. With further enhancements and continuous data collection, such systems could play a significant role in reducing the impact of locust outbreaks on food security in vulnerable regions.

REFERENCES

Abraham Kuriakose, Raiz Badarudheen, & Lalitaditya Charapanjeri. (2023). Swarm Drone System with YOLOv8 Algorithm for Efficient Locust Management in Agricultural Environments. *International Journal of Advanced Research in Science, Communication and Technology*, 177–188. https://doi.org/10.48175/IJARSCT-11430

Anjita, N. A., Indu, J., Thiruvengadam, P., Dixit, V., Rastogi, A., & Arul Malar Kannan, B. S. (2024a). Doppler weather radars as a game changer in desert locust swarm tracking. *Scientific Reports*, *14*(1), 31715. https://doi.org/10.1038/s41598-024-81553-1

Anjita, N. A., Indu, J., Thiruvengadam, P., Dixit, V., Rastogi, A., & Arul Malar Kannan, B. S. (2024b). Doppler weather radars as a game changer in desert locust swarm tracking. *Scientific Reports*, *14*(1), 31715. https://doi.org/10.1038/s41598-024-81553-1

Barnhart, I., Demarco, P., Prasad, P. V. V., Mayor, L., Jugulam, M., & Ciampitti, I. A. (2021). High-resolution unmanned aircraft systems imagery for stay-green characterization in grain sorghum (Sorghum bicolor L.). *Journal of Applied Remote Sensing*, *15*(4), 044501. https://doi.org/10.1117/1.JRS.15.044501

Bernoff, A. J., Culshaw-Maurer, M., Everett, R. A., Hohn, M. E., Strickland, W. C., & Weinburd, J. (2020). Agent-based and continuous models of hopper bands for the Australian plague locust: How resource consumption mediates pulse formation and geometry. *PLoS Computational Biology*, *16*(5), e1007820. https://doi.org/10.1371/journal.pcbi.1007820

Chen, C., Chen, C., Qian, J., Qian, J., Qian, J., Qian, J., Chen, X., Chen, X., Chen, X., Chen, X., Chen, X., Hu, Z., Hu, Z., Sun, J., Sun, J., Wei, S., Wei, S., Xu, K., & Xu, K. (2020). Geographic Distribution of Desert Locusts in Africa, Asia and Europe Using Multiple Sources of Remote-Sensing Data. *Remote Sensing*. https://doi.org/10.3390/rs12213593

Cressman, K. (2016). Desert Locust. In *Biological and Environmental Hazards, Risks, and Disasters* (pp. 87–105). Elsevier. https://doi.org/10.1016/B978-0-12-394847-2.00006-1

Dinku, T., Ceccato, P., Grover-Kopec, E., Lemma, M., Connor, S. J., & Ropelewski, C. F. (2007). Validation of satellite rainfall products over East Africa's complex topography. *International Journal of Remote Sensing*. https://doi.org/10.1080/01431160600954688

Ellenburg, W. L., Ellenburg, W. L., Ellenburg, W. L., Mishra, V., Mishra, V., Roberts, J. B., Roberts, J. B., Roberts, J. B., Roberts, J. B., Limaye, A., Limaye, A., Case, J. L., Case, J. L., Blankenship, C., Blankenship, C., Cressman, K., & Cressman, K. (2021). Detecting Desert Locust Breeding Grounds: A Satellite-Assisted Modeling Approach. *Remote Sensing*. https://doi.org/10.3390/rs13071276

FAO. (2021). *Desert locust crisis 2020-2021*. Food and Agriculture Organization of the United Nations. http://www.fao.org/locusts/en/

FAO. (2022, November 11). *How Somalia used biopesticides to win against desert locusts*. Newsroom. https://www.fao.org/newsroom/story/How-Somalia-used-biopesticides-to-win-against-desert-locusts/?utm_source=chatgpt.com

Finch, E. A., Li, H., Cornelius, A., Styles, J., Beeken, J., Cheng, Y., Wang, G., Qiu, G., & Luke, B. (2024). An updated and validated model for predicting the performance of a biological control agent for the oriental migratory locust. *Pest Management Science*, *80*(2), 442–451. https://doi.org/10.1002/ps.7775

Gebregiorgis, D., Asrat, A., Birhane, E., Tiwari, C., Kiage, L. M., Ramisetty-Mikler, S., Kallam, S., Kabengi, N., Gebrekirstos, A., Wanjiru, S., Mariam, H. G., Fitiwy, I., Haile, M., Enns, C., & Bersaglio, B. (2025). Critical gaps in the global fight against locust outbreaks and addressing emerging challenges. *Npj Sustainable Agriculture*, *3*(1), 29. https://doi.org/10.1038/s44264-025-00068-y

Geng, Y., Zhao, L., Dong, Y., Huang, W., Shi, Y., Ren, Y., & Ren, B. (2020). Migratory Locust Habitat Analysis With PB-AHP Model Using Time-Series Satellite Images. *IEEE Access*, *8*, 166813–166823. IEEE Access. https://doi.org/10.1109/ACCESS.2020.3023264

Gómez, D., Gómez, D. S.-C., Gómez, D., Gómez, D., Salvador, P., Salvador, P., Sanz, J., Sanz, J., Casanova, C., Casanova, C., Casanova, C., Taratiel, D., Taratiel, D., Casanova, J., & Casanova, J. L. (2019). Desert locust detection using Earth observation satellite data in Mauritania. *Journal of Arid Environments*. https://doi.org/10.1016/j.jaridenv.2019.02.005

Gómez, D., Salvador, P., Sanz, J., Casanova, C., Taratiel, D., & Casanova, J. L. (2018). Machine learning approach to locate desert locust breeding areas based on ESA CCI soil moisture. *Journal of Applied Remote Sensing*, *12*(3), 036011. https://doi.org/10.1117/1.JRS.12.036011

Goswami, S., Mondal, S., Johardar, S., & Das, C. B. (2024). Multi-objective function-based optimal path selection for energy efficient routing in MANET using fused locust swarm and dragonfly optimization strategy. *Intelligent Decision Technologies*, 18724981241289763. https://doi.org/10.1177/18724981241289763

Hopkins, M. (2024, November 18). AI and Pest Management: Protecting Yields with Smart Technology. *AgriBusiness Global*. https://www.agribusinessglobal.com/agtech/ai-and-pest-management-protecting-yields-with-smart-technology/

Ian, Goodfellow, Yoshua, Bengio, & Aaron, Courville. (2016). *Deep Learning*. https://www.deeplearningbook.org/

ICRC. (2022, December 14). *Somalia: New swarms of desert locusts pose a threat to farmlands | The ICRC in Somalia*. https://blogs.icrc.org/somalia/2020/12/14/somalia-new-swarms-of-desert-locusts-pose-a-threat-to-farmlands/?utm_source=chatgpt.com

Jiang, H. (2021). *Machine Learning Fundamentals: A Concise Introduction* (1st ed.). Cambridge University Press. https://doi.org/10.1017/9781108938051

Khan, S., Akram, B. A., Zafar, A., Wasim, M., Khurshid, K. S., & Pires, I. M. (2024). LocustLens: Leveraging environmental data fusion and machine learning for desert locust swarm prediction. *PeerJ Computer Science*, *10*, e2420. https://doi.org/10.7717/peerj-cs.2420

Kimathi, E., Tonnang, H. E. Z., Subramanian, S., Cressman, K., Abdel-Rahman, E. M., Tesfayohannes, M., Niassy, S., Torto, B., Dubois, T., Tanga, C. M., Kassie, M., Ekesi, S., Mwangi, D., & Kelemu, S. (2020). Prediction of breeding regions for the desert locust Schistocerca

gregaria in East Africa. *Scientific Reports*, *10*(1), Article 1. https://doi.org/10.1038/s41598-020-68895-2

Klein, I., Klein, I., Oppelt, N., Oppelt, N., Kuenzer, C., & Kuenzer, C. (2021). Application of Remote Sensing Data for Locust Research and Management-A Review. *Insects*. https://doi.org/10.3390/insects12030233

Klein, I., van der Woude, S., Schwarzenbacher, F., Muratova, N., Slagter, B., Malakhov, D., Oppelt, N., & Kuenzer, C. (2022). Predicting suitable breeding areas for different locust species – A multi-scale approach accounting for environmental conditions and current land cover situation. *International Journal of Applied Earth Observation and Geoinformation*, *107*, 102672. https://doi.org/10.1016/j.jag.2021.102672

Landmann, Tobias. (2023). Towards early response to desert locust swarming in eastern Africa by estimating timing of hatching. *Ecological Modelling*, *484*, 110476. https://doi.org/10.1016/j.ecolmodel.2023.110476

Latchininsky, A. V., Latchininsky, A. V., Piou, C., Piou, C., Piou, C., Piou, C., Franc, A., Franc, A., Soti, V., & Soti, V. (2016). Applications of remote sensing to locust management. *Null*. https://doi.org/10.1016/b978-1-78548-105-5.50008-6

LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, *521*(7553), Article 7553. https://doi.org/10.1038/nature14539

Marković, D., Vujičić, D., Tanasković, S., Đorđević, B., Ranđić, S., & Stamenković, Z. (2021). Prediction of Pest Insect Appearance Using Sensors and Machine Learning. *Sensors*, *21*(14), Article 14. https://doi.org/10.3390/s21144846

Maryam Tabar, Jared Gluck, Anchit Goyal, Fei Jiang, Derek Morr, Annalyse Kehs, Dongwon Lee, David P. Hughes, & Amulya Yadav. (2021). A PLAN for Tackling the Locust Crisis in East Africa: Harnessing Spatiotemporal Deep Models for Locust Movement Forecasting. *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, 3595–3604. https://doi.org/10.1145/3447548.3467184

Meynard, C. N., Meynard, C. N., Lecoq, M., Lecoq, M., Lecoq, M., Chapuis, M. P., Chapuis, M.-P., Chapuis, M. P., Piou, C., & Piou, C. (2020). On the relative role of climate change and management in the current desert locust outbreak in East Africa. *Global Change Biology*. https://doi.org/10.1111/gcb.15137

Pramod, A., R, Deeksha., H.C, N., & R Munavalli, Dr. J. (2020). AUTOMATED REAL-TIME LOCUST MANAGEMENT USING ARTIFICIAL INTELLIGENCE. *International Journal of Engineering Applied Sciences and Technology*, *5*(4), 133–138. https://doi.org/10.33564/IJEAST.2020.v05i04.017

Reichstein, M., Camps-Valls, G., Stevens, B., Jung, M., Denzler, J., Carvalhais, N., & Prabhat. (2019). Deep learning and process understanding for data-driven Earth system science. *Nature*, *566*(7743), Article 7743. https://doi.org/10.1038/s41586-019-0912-1

Retkute, R., Thurston, W., Cressman, K., & Gilligan, C. A. (2024). A framework for modelling desert locust population dynamics and large-scale dispersal. *PLOS Computational Biology*, *20*(12), e1012562. https://doi.org/10.1371/journal.pcbi.1012562

Rhodes, K., & Sagan, V. (2022). Integrating Remote Sensing and Machine Learning for Regional-Scale Habitat Mapping: Advances and future challenges for desert locust monitoring. *IEEE Geoscience and Remote Sensing Magazine*, *10*(1), 289–319. IEEE Geoscience and Remote Sensing Magazine. https://doi.org/10.1109/MGRS.2021.3097280

Samil, H. M. O. A., Martin, A., Jain, A. K., Amin, S., & Kahou, S. E. (2021). *Predicting Regional Locust Swarm Distribution with Recurrent Neural Networks* (arXiv:2011.14371). arXiv. https://doi.org/10.48550/arXiv.2011.14371

Sarkar, S. (2020, September 29). *The chemicals used to kill locusts also harm humans and the environment*. Scroll.In. https://scroll.in/article/974249/the-chemicals-used-to-kill-locusts-also-harm-humans-and-the-environment

shannon@smcnamara.com. (2024, April 29). Using AI for Pest Management. *Minor Use Foundation*. https://minorusefoundation.org/ai-for-pest-management/

Shao, Z., Feng, X., Bai, L., Jiao, H., Zhang, Y., Li, D., Fan, H., Huang, X., Ding, Y., Altan, O., & Saleem, N. (2021). Monitoring and Predicting Desert Locust Plague Severity in Asia–Africa Using Multisource Remote Sensing Time-Series Data. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, *14*, 8638–8652. IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing. https://doi.org/10.1109/JSTARS.2021.3104936

Sharma, A. (2014). Locust Control Management: Moving from Traditional to New Technologies Ã¢Â An Empirical Analysis. *Entomology, Ornithology & Herpetology: Current Research*, *4*(1), 1–7. https://doi.org/10.4172/2161-0983.1000141

Showler, A. T., Ould Babah Ebbe, M. A., Lecoq, M., & Maeno, K. O. (2021). Early Intervention against Desert Locusts: Current Proactive Approach and the Prospect of Sustainable Outbreak Prevention. *Agronomy*, *11*(2), Article 2. https://doi.org/10.3390/agronomy11020312

Showler, A. T., Shah, S., Sulaiman, Khan, S., Ullah, S., & Degola, F. (2022). Desert Locust Episode in Pakistan, 2018–2021, and the Current Status of Integrated Desert Locust Management. *Journal of Integrated Pest Management*, *13*(1). https://doi.org/10.1093/jipm/pmab036

Sidra Khan, Beenish Ayesha Akram, Amna Zafar, Muhammad Wasim, Khaldoon S. Khurshid, & Ivan Miguel Pires. (2024). LocustLens: Leveraging environmental data fusion and machine learning for desert locust swarm prediction. *PeerJ Computer Science*, *10*, e2420. https://doi.org/10.7717/peerj-cs.2420

Sokame, B. M., Agboka, K. M., Kimathi, E., Mudereri, B. T., Abdel-Rahman, E. M., Landmann, T., Rwaheru, M. M., Abdalla, O., Mafabi, M. M., Lubango, L. M., & Tonnang, H. E. Z. (2024). An Integrated Assessment Approach for Socio-Economic Implications of the Desert Locust in Eastern Africa. *Earth's Future*, *12*(4), e2023EF003841. https://doi.org/10.1029/2023EF003841

Sun, R., Huang, W., Dong, Y., Zhao, L., Zhang, B., Ma, H., Geng, Y., Ruan, C., Xing, N., Chen, X., & Li, X. (2022). Dynamic Forecast of Desert Locust Presence Using Machine Learning with a Multivariate Time Lag Sliding Window Technique. *Remote Sensing*, *14*(3), Article 3. https://doi.org/10.3390/rs14030747

Tucker. (1985). Satellite remote sensing of total herbaceous biomass production in the senegalese sahel: 1980–1984. *Remote Sensing of Environment*, *17*(3), 233–249. https://doi.org/10.1016/0034-4257(85)90097-5

Wight, A. (2024). *How Can AI Help Predict Locust Outbreaks In West Africa?* Forbes. https://www.forbes.com/sites/andrewwight/2024/05/20/how-can-ai-help-predict-locust-outbreaks-in-west-africa/

Word Ries, M., Adriaansen, C., Aldobai, S., Berry, K., Bal, A. B., Catenaccio, M. C., Cigliano, M. M., Cullen, D. A., Deveson, T., Diongue, A., Foquet, B., Hadrich, J., Hunter, D., Johnson, D. L., Pablo Karnatz, J., Lange, C. E., Lawton, D., Lazar, M., Latchininsky, A. V., … Cease, A. (2024). Global perspectives and transdisciplinary opportunities for locust and grasshopper pest management and research. *Journal of Orthoptera Research*, *33*(2), 169–216. https://doi.org/10.3897/jor.33.112803

Ye, S., Lu, S., Bai, X., & Gu, J. (2020). ResNet-Locust-BN Network-Based Automatic Identification of East Asian Migratory Locust Species and Instars from RGB Images. *Insects*, *11*(8), Article 8. https://doi.org/10.3390/insects11080458

Yusuf, I. S., Mukhtar Opeyemi Yusuf, Kobby Panford-Quainoo, & Arnu Pretorius. (2024, March 11). *A Geospatial Approach to Predicting Desert Locust Breeding Grounds in Africa*. arXiv.Org. https://arxiv.org/abs/2403.06860v2

Yusuf, I. S., Tessera, K., Tumiel, T., Slim, Z., Kerkeni, A., Nevo, S., & Pretorius, A. (2022). *On pseudo-absence generation and machine learning for locust breeding ground prediction in Africa* (arXiv:2111.03904). arXiv. https://doi.org/10.48550/arXiv.2111.03904

Zhang, W., Huang, H., Sun, Y., & Wu, X. (2022). AgriPest-YOLO: A rapid light-trap agricultural pest detection method based on deep learning. *Frontiers in Plant Science*, *13*, 1079384. https://doi.org/10.3389/fpls.2022.1079384

APPENDICES

Appendix A: Import Libraries

```
from flask import Flask, request, jsonify, send_from_directory,
send_file, make_response

# Initialize Flask with correct static folder for frontend
app = Flask(__name__, static_folder='../frontend/public',
static_url_path='/')
from flask_cors import CORS
from flask_jwt_extended import JWTManager, create_access_token,
jwt_required, get_jwt_identity, get_jwt, get_jwt_identity
import mysql.connector
from mysql.connector import Error
import bcrypt
from datetime import datetime, timedelta
import os
import uuid
from dotenv import load_dotenv
import numpy as np
import pandas as pd
import json
import joblib
import datetime
import re
from functools import lru_cache
from bs4 import BeautifulSoup
```

Appendix B: Database Iinitialization

```
# Database configuration
db_config = {
```

```
    'host': os.getenv('DB_HOST', 'localhost'),
    'user': os.getenv('DB_USER', 'root'),
    'password': os.getenv('DB_PASSWORD', ''),
    'database': os.getenv('DB_NAME', 'ml_project')
}
```

Appendix C: Model Loading

```
# Load the model and target encodings

MODEL_DIR = os.path.join(os.path.dirname(os.path.dirname(__file__)),
'ml', 'models')
MODEL_PATH = os.path.join(MODEL_DIR, 'random_forest.pkl')

# Load target encodings from the training data
def load_target_encodings():
    try:
        # Load the original training data
        data_path =
os.path.join(os.path.dirname(os.path.dirname(__file__)), 'ml', 'data',
'raw', 'locust_dataset.csv')
        print(f"Loading dataset from: {data_path}")
        df = pd.read_csv(data_path)

        # Preprocess the data similar to training
        df['REGION'] = df['REGION'].str.strip().str.upper()
        df['COUNTRYNAME'] = df['COUNTRYNAME'].str.strip().str.upper()
        df['LOCUSTPRESENT'] = df['LOCUSTPRESENT'].str.strip().str.upper()
        df['LOCUSTPRESENT'] = df['LOCUSTPRESENT'].map({'YES': 1, 'NO':
0})

        # Calculate target encodings
        country_target_means =
df.groupby('COUNTRYNAME')['LOCUSTPRESENT'].mean()
        region_target_means =
df.groupby('REGION')['LOCUSTPRESENT'].mean()

        return country_target_means, region_target_means
    except Exception as e:
        print(f"Error loading target encodings: {e}")
        # Return default values if loading fails
        return pd.Series(), pd.Series()

# Load model and target encodings at startup
model = None
country_encodings, region_encodings = pd.Series(), pd.Series()

# Try to load the default model, but don't fail if it's not available
try:
    # First try to load the default model
    if os.path.exists(MODEL_PATH):
```

```
            model = joblib.load(MODEL_PATH)
            print("Default ML model loaded successfully.")
        else:
            print(f"Warning: Default model not found at {MODEL_PATH}")

        # Always try to load the target encodings
        country_encodings, region_encodings = load_target_encodings()
        if not country_encodings.empty and not region_encodings.empty:
            print("Target encodings loaded successfully.")
        else:
            print("Warning: Could not load target encodings or they are
empty")

    except Exception as e:
        print(f"Warning: Error during model/encoding initialization:
{str(e)}")
        print("The application will continue but some features may not work
correctly.")
        print("Please check the model files in the ml/models directory.")

    def init_db():
        """Initialize the database with required tables if they don't
exist."""
        conn = None
        cursor = None
        try:
            conn = mysql.connector.connect(**db_config)
            cursor = conn.cursor()

            # Create users table if it doesn't exist
            cursor.execute('''
            CREATE TABLE IF NOT EXISTS users (
                id INT AUTO_INCREMENT PRIMARY KEY,
                username VARCHAR(80) UNIQUE NOT NULL,
                email VARCHAR(120) UNIQUE NOT NULL,
                password_hash VARCHAR(255) NOT NULL,
                created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
            ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_unicode_ci;
            ''')

            # Create predictions table if it doesn't exist
            cursor.execute('''
            CREATE TABLE IF NOT EXISTS predictions (
                id INT AUTO_INCREMENT PRIMARY KEY,
                user_id INT NOT NULL,
                region VARCHAR(100) NOT NULL,
                country_name VARCHAR(100) NOT NULL,
                prediction_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                locust_present TINYINT(1) NOT NULL,
                probability FLOAT,
```

```
            FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
            INDEX (user_id, prediction_date)
        ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_unicode_ci;
        ''')

        # Create blog_posts table if it doesn't exist
        cursor.execute('''
        CREATE TABLE IF NOT EXISTS blog_posts (
            id INT AUTO_INCREMENT PRIMARY KEY,
            title VARCHAR(255) NOT NULL,
            content LONGTEXT NOT NULL,
            region VARCHAR(100),
            country VARCHAR(100),
            date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
            author VARCHAR(100),
            tags TEXT,
            image_url VARCHAR(255),
            FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
        ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_unicode_ci;
        ''')
```

user_id INT NOT NULL,

```
        conn.commit()
        print("Database tables verified/created successfully")

    except mysql.connector.Error as e:
        print(f"Error initializing database: {e}")
        if conn and conn.is_connected():
            conn.rollback()
        raise
    finally:
        if cursor:
            cursor.close()
        if conn and conn.is_connected():
            conn.close()
```

Appendix D: Get Database Connection

```
def get_db_connection():
    """Get a database connection and ensure tables exist."""
    try:
        connection = mysql.connector.connect(**db_config)
        print('Successfully connected to database')
        return connection
    except mysql.connector.Error as e:
        print(f'Error connecting to database: {e}')
        raise
```

Appendix E: Sing Up

```
# User Registration
@app.route('/api/register', methods=['POST'])
def register():
    try:
        data = request.get_json()
        full_name = data.get('full_name')
        email = data.get('email')
        password = data.get('password')
        security_question = data.get('security_question')
        security_answer = data.get('security_answer')

        if not all([full_name, email, password, security_question,
security_answer]):
            return jsonify({'error': 'All fields are required'}), 400

        # Hash password
        hashed_password = bcrypt.hashpw(password.encode('utf-8'),
bcrypt.gensalt())

        conn = get_db_connection()
        cursor = conn.cursor()

        # Check if email already exists
        cursor.execute('SELECT id FROM users WHERE email = %s', (email,))
        if cursor.fetchone():
            return jsonify({'error': 'Email already registered'}), 400

        # Insert new user with security question and answer
        cursor.execute(
            'INSERT INTO users (full_name, email, password,
security_question, security_answer) VALUES (%s, %s, %s, %s, %s)',
            (full_name, email, hashed_password, security_question,
security_answer)
        )
        conn.commit()

        return jsonify({'message': 'User registered successfully'}), 201

    except Exception as e:
        return jsonify({'error': str(e)}), 500
    finally:
        cursor.close()
        conn.close()
```

Appendix F: User Login

```
# User Login
@app.route('/api/login', methods=['POST'])
def login():
```

```
    conn = None
    cursor = None
    try:
        data = request.get_json()
        email = data.get('email')
        password = data.get('password')

        if not all([email, password]):
            return jsonify({'error': 'Email and password are required'}),
400

        conn = get_db_connection()
        cursor = conn.cursor(dictionary=True)

        # Get user
        cursor.execute('SELECT id, email, password, full_name FROM users
WHERE email = %s', (email,))
        user = cursor.fetchone()

        if not user or not bcrypt.checkpw(password.encode('utf-8'),
user['password'].encode('utf-8')):
            return jsonify({'error': 'Invalid email or password'}), 401

        # Use the user ID as the identity (must be a string)
        user_id = str(user['id'])
        user_data = {
            'id': user_id,
            'email': user['email'],
            'full_name': user['full_name']
        }

        # Create access token with user ID as identity
        # Additional user data will be included via the
additional_claims_loader
        access_token = create_access_token(identity=user_id)

        # Return token and user data
        response_data = {
            'access_token': access_token,
            'user': user_data,
            'message': 'Login successful',
            'status': 'success'
        }
        print(f"[DEBUG] Login response: {response_data}")
        return jsonify(response_data), 200

    except Exception as e:
        print(f"Login error: {str(e)}")  # Add debug logging
        return jsonify({'error': str(e)}), 500
    finally:
        if cursor:
```

```
            cursor.close()
        if conn:
            conn.close()
```

Appendix G: Make Prediction

```
# Make prediction
        prediction = model_to_use.predict(input_data)
        probabilities = model_to_use.predict_proba(input_data)
        print("Prediction:", prediction)
        print("Probabilities:", probabilities)
        # Return prediction and probability
        return jsonify({
            'prediction': 'yes' if prediction[0] == 1 else 'no',
            'probability': float(probabilities[0][1]),  # Probability of
class 1 (yes)
            'matched_region': data['REGION'].strip().upper(),
            'model_used': model_name
        })
    except Exception as e:
        print("Error during prediction:", str(e))
        return jsonify({'error': str(e)}), 500

# Global error handler for /api/predict
@app.errorhandler(422)
def handle_422(err):
    messages = ['An unprocessable entity error occurred.']
    if hasattr(err, 'data') and 'messages' in err.data:
        messages = err.data['messages']
    print(f"422 Error: {messages}")

# Save Prediction
@app.route('/api/save_prediction', methods=['POST'])
def save_prediction():
    cursor = None
    connection = None
    try:
        data = request.get_json()
        print('Received data:', data)

        # Get user email from Authorization header
        auth_header = request.headers.get('Authorization')
        user_email = None
        if auth_header:
            try:
                user_data = json.loads(auth_header)
                if user_data.get('isLoggedIn'):
                    user_email = user_data.get('email')
            except Exception as e:
                print('Error parsing auth header:', str(e))
```

```
                pass

        if not user_email:
            return jsonify({'error': 'Not authenticated'}), 401

        connection = get_db_connection()
        if not connection:
            return jsonify({'error': 'Database connection failed'}), 500

        # First get user_id from email
        cursor = connection.cursor(dictionary=True)
        cursor.execute('SELECT id FROM users WHERE email = %s',
(user_email,))
        user = cursor.fetchone()
        if not user:
            return jsonify({'error': 'User not found'}), 404
        user_id = user['id']

        cursor = connection.cursor()
        sql = """
            INSERT INTO predictions
            (user_id, region, country_name, start_year, start_month,
            soil_moisture, tmax, ppt, locust_present, probability)
            VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
        """
        values = (
            user_id,
            data['region_name'],
            data['country_name'],
            data['start_year'],
            data['start_month'],
            data['soil_moisture_percent'],
            data['temperature_celsius'],
            data['precipitation_mm'],
            data['prediction_result'],
            data.get('probability', None)
        )

        print('Executing SQL with values:', values)
        cursor.execute(sql, values)
        connection.commit()

        return jsonify({
            'message': 'Prediction saved successfully',
            'id': cursor.lastrowid
        }), 200
    except mysql.connector.Error as e:
        print('Database error:', str(e))
        return jsonify({'error': f'Database error: {str(e)}'}), 500
    except Exception as e:
        print('Error saving prediction:', str(e))
```

```
        return jsonify({'error': str(e)}), 500
    finally:
        if cursor:
            cursor.close()
        if connection:
            connection.close()
```

Appendix H: Get a User's Predictions

```
# Get User's Predictions
@app.route('/api/predictions/<int:prediction_id>', methods=['DELETE'])
@jwt_required()
def delete_prediction(prediction_id):
    conn = None
    cursor = None
    try:
        # Get the current user ID from JWT
        user_id = get_jwt_identity()

        conn = get_db_connection()
        cursor = conn.cursor(dictionary=True)

        # Check if the prediction exists and belongs to the user
        cursor.execute('SELECT id FROM predictions WHERE id = %s AND
user_id = %s', (prediction_id, user_id))
        prediction = cursor.fetchone()

        if not prediction:
            return jsonify({
                'status': 'error',
                'error': 'Prediction not found or access denied'
            }), 404
```

Appendix I: Delete Prediction

```
        # Delete the prediction
        cursor.execute('DELETE FROM predictions WHERE id = %s',
(prediction_id,))
        conn.commit()

        return jsonify({
            'status': 'success',
            'message': 'Prediction deleted successfully'
        })

    except Exception as e:
        if conn:
            conn.rollback()
        print(f"Error deleting prediction: {str(e)}")
        return jsonify({
            'status': 'error',
```

```
            'error': 'Failed to delete prediction',
            'details': str(e)
        }), 500
    finally:
        if cursor:
            cursor.close()
        if conn and conn.is_connected():
            conn.close()

@app.route('/api/predictions/<int:prediction_id>', methods=['GET'])
@jwt_required()
def get_prediction(prediction_id):
    conn = None
    cursor = None
    try:
        print(f"[DEBUG] Fetching prediction with ID: {prediction_id}")

        # Get the current user ID from JWT
        user_id = get_jwt_identity()
        print(f"[DEBUG] Current user ID: {user_id}")

        if not user_id:
            print("[ERROR] No user ID in token")
            return jsonify({'status': 'error', 'message':
'Unauthorized'}), 401

        try:
            conn = get_db_connection()
            cursor = conn.cursor(dictionary=True)

            # First, check if the prediction exists and user has access
            query = """
                SELECT p.*, u.full_name, u.email
                FROM predictions p
                JOIN users u ON p.user_id = u.id
                WHERE p.id = %s
            """
            print(f"[DEBUG] Executing query with prediction_id:
{prediction_id}")
            cursor.execute(query, (prediction_id,))
            prediction = cursor.fetchone()

            if not prediction:
                print(f"[DEBUG] No prediction found with ID:
{prediction_id}")
                return jsonify({'status': 'error', 'message': 'Prediction
not found'}), 404

            print(f"[DEBUG] Found prediction: {prediction}")

            # Check if user has permission to view this prediction
```

```
            # Since we don't have roles, only allow users to view their
own predictions
            if str(prediction['user_id']) != str(user_id):
                print(f"[DEBUG] Access denied for user {user_id} to
prediction {prediction_id}")
                return jsonify({
                    'status': 'error',
                    'message': 'Access denied. You can only view your own
predictions.'
                }), 403

            # Convert Decimal to float for JSON serialization
            result = {}
            for key, value in prediction.items():
                if hasattr(value, 'to_eng_string'):
                    result[key] = float(value)
                else:
                    result[key] = value

            print(f"[DEBUG] Returning prediction data")
            return jsonify({
                'status': 'success',
                'data': result
            })

        except Exception as db_error:
            print(f"[ERROR] Database error: {str(db_error)}")
            print(f"[ERROR] Query: {query}")
            print(f"[ERROR] Params: ({prediction_id},)")
            raise

    except Exception as e:
        print(f"[ERROR] Failed to fetch prediction: {str(e)}")
        import traceback
        traceback.print_exc()
        return jsonify({
            'status': 'error',
            'message': 'Failed to fetch prediction details',
            'debug': str(e)
        }), 500

    finally:
        if cursor:
            cursor.close()
        if conn and conn.is_connected():
            conn.close()
```

Appendix J: Forgot Password

```
# Forgot Password - Step 1: Get Security Question
@app.route('/api/auth/forgot-password', methods=['POST'])
```

```
def forgot_password():
    print("\n=== Forgot Password Request ===")
    print(f"Request Headers: {dict(request.headers)}")
    print(f"Request Data: {request.get_data()}")

    conn = None
    cursor = None
    try:
        # Get data from JSON request
        data = request.get_json()
        print(f"Parsed JSON Data: {data}")

        if not data:
            print("Error: No data provided")
            return jsonify({'error': 'No data provided'}), 400

        email = data.get('email')
        print(f"Looking up email: {email}")

        if not email:
            print("Error: Email is required")
            return jsonify({'error': 'Email is required'}), 400

        conn = get_db_connection()
        cursor = conn.cursor(dictionary=True)

        # First, check if any users exist in the database
        cursor.execute('SELECT COUNT(*) as count FROM users')
        user_count = cursor.fetchone()['count']
        print(f"Total users in database: {user_count}")

        # Debug: List all users in the database
        if user_count > 0:
            cursor.execute('SELECT id, email FROM users LIMIT 10')
            all_users = cursor.fetchall()
            print("First 10 users in database:")
            for u in all_users:
                print(f"ID: {u['id']}, Email: {u['email']}")

        # Get the user's security question (case-insensitive search)
        query = 'SELECT id, email, security_question FROM users WHERE
LOWER(email) = LOWER(%s)'
        print(f"Executing query: {query} with email: {email}")
        cursor.execute(query, (email,))
        user = cursor.fetchone()

        if not user:
            print(f"No user found with email: {email}")
            return jsonify({'error': 'No account found with this
email'}), 404
```

```
            print(f"Found user: ID={user['id']}, Email={user['email']}")

            # Return the security question in JSON format
            response = {
                'success': True,
                'security_question': user['security_question']
            }
            print(f"Returning response: {response}")
            return jsonify(response)

        except Exception as e:
            print(f"Error in forgot-password: {str(e)}")
            return jsonify({
                'error': 'An error occurred while processing your request',
                'details': str(e)
            }), 500
        finally:
            if cursor:
                cursor.close()
            if conn:
                conn.close()

    # Forgot Password - Step 2: Verify Security Answer
    @app.route('/api/auth/verify-answer', methods=['POST'])
    def verify_answer():
        conn = None
        cursor = None
        try:
            # Get data from JSON request
            data = request.get_json()
            if not data:
                return jsonify({'error': 'No data provided'}), 400

            email = data.get('email')
            answer = data.get('answer')

            if not all([email, answer]):
                return jsonify({'error': 'Email and answer are required'}),
    400

            conn = get_db_connection()
            cursor = conn.cursor(dictionary=True)

            # Verify the security answer (case-insensitive)
            cursor.execute(
                'SELECT id FROM users WHERE email = %s AND
    LOWER(security_answer) = LOWER(%s)',
                (email, answer.strip())
            )
            user = cursor.fetchone()
```

```
        if not user:
            return jsonify({'error': 'Incorrect answer. Please try
again.'}), 401

        # Return success response
        return jsonify({
            'success': True,
            'message': 'Answer verified successfully'
        })

    except Exception as e:
        print(f"Error in verify-answer: {str(e)}")
        return jsonify({
            'error': 'An error occurred while verifying your answer',
            'details': str(e)
        }), 500
    finally:
        if cursor:
            cursor.close()
        if conn:
            conn.close()

# Forgot Password - Step 3: Reset Password
@app.route('/api/auth/reset-password', methods=['POST'])
def reset_password():
    print("\n=== Reset Password Request ===")
    print(f"Request Headers: {dict(request.headers)}")
    print(f"Request Data: {request.get_data()}")

    conn = None
    cursor = None
    try:
        # Get data from JSON request
        data = request.get_json()
        print(f"Parsed JSON Data: {data}")

        if not data:
            print("Error: No data provided")
            return jsonify({'error': 'No data provided'}), 400

        email = data.get('email')
        new_password = data.get('new_password')

        print(f"Email: {email}, Password Length: {len(new_password) if
new_password else 0}")

        if not all([email, new_password]):
            print("Error: Missing email or password")
            return jsonify({'error': 'Email and new password are
required'}), 400
```

```
        if len(new_password) < 8:
            print("Error: Password too short")
            return jsonify({'error': 'Password must be at least 8
characters long'}), 400

        conn = get_db_connection()
        cursor = conn.cursor(dictionary=True)

        # First, check if user exists
        cursor.execute('SELECT id FROM users WHERE email = %s', (email,))
        user = cursor.fetchone()
        print(f"User lookup result: {user}")

        if not user:
            print(f"Error: No user found with email: {email}")
            return jsonify({'error': 'User not found'}), 404

        # Hash the new password
        hashed_password = bcrypt.hashpw(new_password.encode('utf-8'),
bcrypt.gensalt())
        print(f"Hashed password: {hashed_password}")

        # Update the password
        try:
            cursor.execute(
                'UPDATE users SET password = %s WHERE email = %s',
                (hashed_password, email)
            )
            print(f"Password update query executed. Rows affected:
{cursor.rowcount}")

            if cursor.rowcount == 0:
                print("Error: No rows were updated")
                return jsonify({'error': 'Failed to update password. No
changes made.'}), 500

            conn.commit()
            print("Database changes committed successfully")

            # Return success response
            response = {
                'success': True,
                'message': 'Password updated successfully. You can now
log in with your new password.'
            }
            print(f"Returning success response: {response}")
            return jsonify(response)

        except Exception as e:
            print(f"Database error during password update: {str(e)}")
            if conn:
```

```
                conn.rollback()
            raise

    except Exception as e:
        print(f"Error in reset-password: {str(e)}")
        if conn:
            conn.rollback()
        return jsonify({'error': 'An error occurred while resetting your
password'}), 500
    finally:
        if cursor:
            cursor.close()
        if conn:
            conn.close()
```

Title of Thesis	
Page or Chapters	**COMMENTS**

Signature _____ Date: _____/08/2025